SDET AJAY

sdetajay.com

API Testing

Manual & Automation

The practical guide for SDETs: from Postman to Rest Assured, framework design and CI/CD.

```
given()
    .spec(requestSpec)
    .body(payload)
.when()
    .post("/customers")
.then()
    .statusCode(201);
```

201 Created

By SDET AJAY

Think Switch, Think SDET AJAY.

Manual API testing | Postman | Rest Assured | Framework design | CI/CD | Interviews

API Testing: Manual & Automation

The practical guide for SDETs: from Postman to Rest Assured, framework design and CI/CD.

Written by **SDET AJAY** | sdetajay.com
Edition 1.0 | September 2026

Copyright © 2026 SDET AJAY. All rights reserved. No part of this book may be reproduced or distributed without prior permission from the author.

All API endpoints, hostnames, payloads and credentials in this book are fictional examples (for instance `qa.example.com`). Library versions shown are illustrative: always check the current stable releases of Rest Assured, TestNG, Jackson and other tools before you use them. Postman, Jenkins, Rest Assured and other product names are trademarks of their respective owners and are used here for identification only.

BEFORE YOU START

About This Book

Everything an SDET needs to test an API by hand, automate it properly and explain it confidently in an interview.

Who this book is for

Manual testers moving into automation, automation engineers who want stronger API and framework fundamentals, and SDET candidates preparing for senior QA automation interviews. You should be comfortable reading basic Java. Everything else is explained from first principles.

How the book is organised

Part	What you will learn
I. Foundations	What API testing is, HTTP and REST essentials, authentication and API security basics.
II. Manual API testing	Postman workflow, scripting and Newman, designing test cases, debugging failures and reporting defects.
III. Automation with Rest Assured	Setup, requests, validations, authentication, API chaining, payloads, JSON and schema validation, in 38 worked scenarios.
IV. Framework, CI/CD and senior scenarios	Layered framework design, environments, Jenkins, Extent reports, failed-test reruns, flaky APIs and faster suites.
V. Interview toolkit	Whiteboard topics, rapid-fire questions, cheat sheets, templates and a glossary.

How each scenario is written

The 50 automation scenarios in Parts III and IV follow one pattern that mirrors how you should answer in an interview: a crisp one-liner first, then the depth, the code and the trap to avoid.

> **SHORT ANSWER**
> The one-sentence answer you would say first in the interview room.

> **INTERVIEW TIP**
> The extra point that separates a good answer from a great one.

> **WATCH OUT**
> A common mistake, a version-specific gotcha or a trap interviewers like to probe.

Running example. Most examples use a fictional customer and payment API at `https://qa.example.com`. Replace the paths, fields and status codes with those of your own application. Code is written for Java 17, TestNG, Rest Assured 5.x and Jackson.

Contents

About This Book 3

Part I: Foundations 5

1. What Is API Testing? . . . 6
2. HTTP and REST Essentials . . . 9
3. Authentication and Security Basics . . . 12

Part II: Manual API Testing 14

4. Manual API Testing with Postman . . . 15
5. Designing API Test Cases . . . 18
6. Debugging Failures and Reporting Defects . . . 21

Part III: Automation with Rest Assured 23

7. Rest Assured Foundations . . . 24
8. Requests, Responses and Validations . . . 28
9. Authentication in Automation . . . 33
10. Dynamic Data and API Chaining . . . 38
11. Payload Design and Validation . . . 41
12. JSON and Schema Validation . . . 45

Part IV: Framework, CI/CD and Senior Scenarios 48

13. Framework Architecture . . . 49
14. CI/CD, Logging and Reporting . . . 54
15. Senior-Level Scenarios . . . 58

Part V: Interview Toolkit 62

16. The Interview Toolkit . . . 63

Appendix A: Quick Reference . . . 66

Appendix B: Templates . . . 67

Appendix C: Glossary . . . 68

Keep Practising . . . 69

PART I

Foundations

What API testing is, how HTTP and REST really work, and how authentication and security shape every test you write.

CHAPTER 1

What Is API Testing?

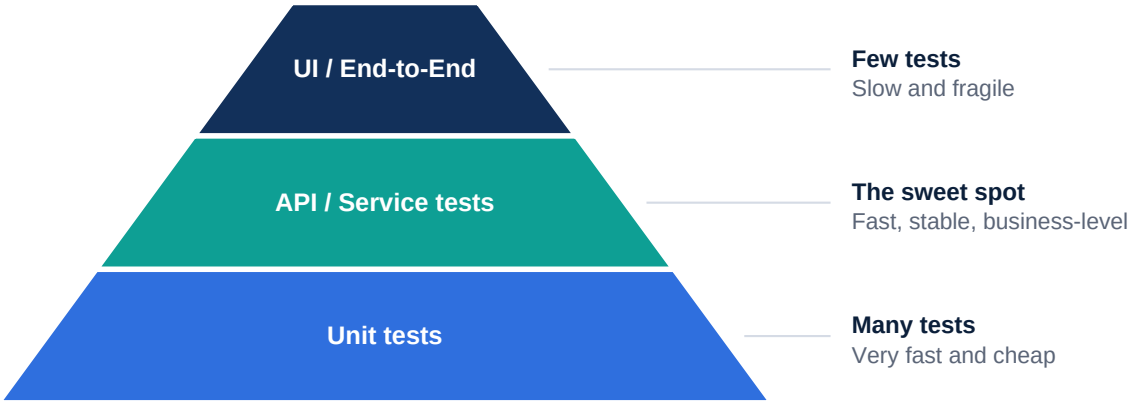
An API is a contract between two pieces of software. API testing checks that the contract holds under every condition that matters.

An **Application Programming Interface (API)** lets one system request data or actions from another without knowing how it is implemented. When a mobile app shows your account balance, it calls an API. When a checkout page charges a card, it calls an API. **API testing** sends requests directly to those interfaces and verifies the responses, the data changes behind them and the behaviour under bad input, missing permissions and heavy load, without going through a user interface.

Why test at the API layer?

- **Speed.** No browser to launch and no page to render. An API test typically runs in milliseconds to seconds.
- **Stability.** There are no locators, animations or visual timing issues, so far fewer false failures.
- **Earlier feedback.** APIs exist before the UI does. Testing them early shifts defects left, where they are cheapest to fix.
- **Deeper coverage.** The UI often blocks invalid input. The API is where you can send a negative amount, a missing field or another user's ID.
- **Business logic lives here.** Validations, calculations, permissions and integrations are implemented in the service layer.
- **CI friendly.** Lightweight, headless and parallelisable.

Where API tests fit: the test pyramid



Unit tests protect individual methods, UI tests protect a few critical journeys, and API tests protect the business rules in between. A healthy suite has most of its functional coverage at the API layer.

What do we verify?

Area	What you check	Example
Functional	Correct behaviour for valid input	POST /customers returns 201 and the customer can be fetched
Negative and validation	Bad input is rejected with the right status and message	Missing mobile number returns 400 with a clear error
Authentication and authorization	Who may call what	User A cannot read User B's account (403)
Contract and schema	Shape, types and required fields	id is an integer, name is a string, no unexpected nulls
Data integrity	Side effects in the database or downstream systems	A payment debits the account exactly once
Reliability	Timeouts, retries, idempotency, error handling	A retried POST does not create a duplicate
Performance	Response time and throughput	95% of calls complete within the agreed SLA
Security	Injection, data exposure, rate limiting	The password field never appears in a response

Types of APIs you will meet

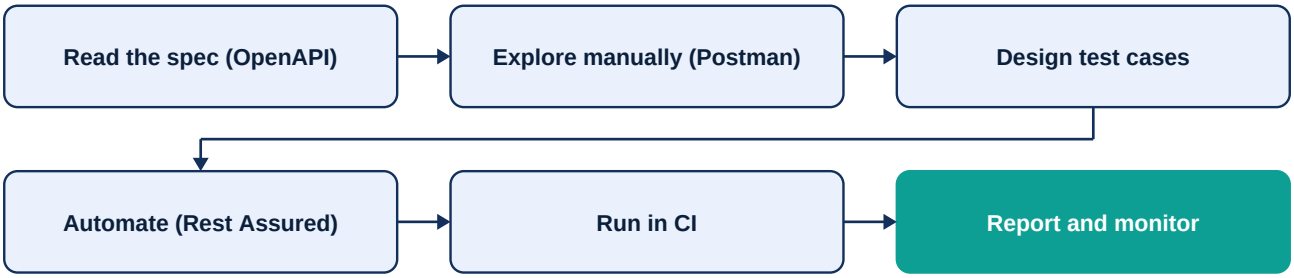
Style	Format	Typical contract	Notes for testers
REST	JSON (sometimes XML) over HTTP	OpenAPI / Swagger	The focus of this book. Resources, HTTP verbs and status codes.
SOAP	XML envelope over HTTP	WSDL	Strict contract, common in banking and legacy enterprise. SoapUI is popular.
GraphQL	JSON, single endpoint	Schema	Clients choose fields. Test queries, mutations and error arrays (often HTTP 200).
gRPC	Protocol Buffers over HTTP/2	.proto files	Binary and typed. Needs specialised tooling.

Manual or automated?

Both. Manual testing is how you *learn* an API. Automation is how you *protect* it.

Prefer manual when	Prefer automation when
Exploring a new or changing endpoint for the first time	The check must run on every build or deployment (regression, smoke)
Investigating a defect and experimenting quickly	The same flow needs many data combinations
Verifying documentation against real behaviour	You need contract, schema and status-code checks at scale
One-off data setup or an ad hoc production check	Multi-step workflows (create, update, verify, delete) must be repeatable

The API testing lifecycle



INTERVIEW TIP

Interviewers often ask, "Why do you prefer API automation over UI automation?" Answer with speed, stability, earlier feedback and coverage of negative cases, then add that you still keep a thin layer of UI tests for critical end-to-end journeys.

CHAPTER 2

HTTP and REST Essentials

You cannot test what you cannot read. Every API test is a conversation in HTTP.

Anatomy of a request and response

A **request** has a method, a URL, headers and an optional body. A **response** has a status line, headers and an optional body.

```
POST /customers HTTP/1.1
Host: qa.example.com
Content-Type: application/json
Accept: application/json
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...

{"name": "Ajay", "mobile": "9876543210"}
```

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /customers/C101

{"customerId": "C101", "status": "SUCCESS"}
```

Anatomy of a URL

```
https://qa.example.com:443/customers/101/orders?status=OPEN&page=2
```

Part	Value	Meaning
Scheme	`https`	Protocol. Always test over HTTPS outside local development.
Host and port	`qa.example.com:443`	The server. This is your base URI and changes per environment.
Path	`/customers/101/orders`	Identifies the resource. Nouns, not verbs.
Path parameter	`101`	A variable inside the path that identifies one resource.
Query parameters	`status=OPEN&page=2`	Optional filters, sorting and pagination. Not part of the resource identity.

HTTP methods

Method	Purpose	Safe	Idempotent	Typical success
GET	Read a resource or a list	Yes	Yes	200
POST	Create a resource or trigger an action	No	No	201 (or 200 / 202)
PUT	Replace a resource completely	No	Yes	200 or 204
PATCH	Partially update a resource	No	Not guaranteed	200 or 204
DELETE	Remove a resource	No	Yes	204 (or 200)

Method	Purpose	Safe	Idempotent	Typical success
HEAD	Like GET, headers only	Yes	Yes	200
OPTIONS	Discover allowed methods	Yes	Yes	200 or 204

Safe means the request does not change server state. **Idempotent** means that sending the same request several times leaves the server in the same state as sending it once. The *response* may still differ: a second DELETE usually returns 404 because the resource is already gone, yet the server state is unchanged.

> **INTERVIEW TIP**
> PUT vs PATCH vs POST is a classic. PUT replaces the whole resource at a URI the client knows. PATCH changes only the fields you send. POST asks the server to create or process something and is not idempotent, which is why payment APIs add an idempotency key so a retry cannot charge twice.

Status codes every tester must know

Code	Meaning	What to check as a tester
200 OK	Request succeeded	Body matches the contract and the data is correct.
201 Created	A resource was created	Location header or ID returned; resource retrievable afterwards.
202 Accepted	Accepted for asynchronous processing	Poll the status endpoint until the job completes.
204 No Content	Success with no body	Body really is empty; state changed as expected.
304 Not Modified	Cached copy is still valid	Conditional GET with ETag or If-Modified-Since.
400 Bad Request	Malformed or invalid request	Clear, consistent error message; no server stack trace.
401 Unauthorized	Not authenticated (missing or invalid credentials)	Missing, expired and tampered tokens are all rejected.
403 Forbidden	Authenticated but not permitted	Role and ownership rules are enforced.
404 Not Found	Resource or route does not exist	Unknown IDs and deleted resources; wrong path.
405 Method Not Allowed	Method not supported on this URL	Allow header lists the valid methods.
409 Conflict	Conflicts with current state	Duplicate creation, version or state conflicts.
415 Unsupported Media Type	Content-Type not accepted	Wrong or missing Content-Type is rejected cleanly.
422 Unprocessable Content	Well-formed but semantically invalid	Business-rule validation errors.
429 Too Many Requests	Rate limit exceeded	Retry-After header; limits enforced fairly.
500 Internal Server Error	Unhandled server failure	Always a defect worth reporting, whatever the input.
502 / 503 / 504	Gateway, availability or timeout issue	Environment or dependency problem; retry only if transient.

Headers that matter

Header	Direction	Purpose
Content-Type	Request and response	Format of the body being sent (for example application/json).
Accept	Request	Format the client wants back.
Authorization	Request	Credentials: Basic, Bearer token, and so on.
Location	Response	URI of a newly created resource (with 201) or a redirect target.
Set-Cookie / Cookie	Response / Request	Session state. Check the Secure, HttpOnly and SameSite attributes.
Cache-Control, ETag	Response	Caching and conditional requests.
Retry-After	Response	How long to wait after 429 or 503.
X-Request-Id / correlation ID	Both	Trace one call across services and logs. Invaluable when reporting defects.

REST in five lines

- **Resources, not actions.** Use `/customers/101`, not `/getCustomer?id=101`.
- **Uniform use of HTTP.** The method expresses the action and the status code expresses the outcome.
- **Stateless.** Every request carries everything the server needs, typically a token. The server keeps no client session between calls.
- **Representations.** The same resource can be delivered as JSON or XML, negotiated with Accept and Content-Type.
- **Cacheable and layered.** Responses declare whether they can be cached, and proxies or gateways may sit in between.

> **WATCH OUT**
> Some APIs return HTTP 200 with an error inside the body (for example {"status":"FAILED"}). Read the contract, then assert *both* the status code and the body. A green status code alone proves very little.

CHAPTER 3

Authentication and Security Basics

Most real-world API defects are not about the happy path. They are about who is allowed to do what.

Authentication vs authorization

	Authentication (AuthN)	Authorization (AuthZ)
Question	Who are you?	What are you allowed to do?
Failure status	401 Unauthorized	403 Forbidden
Examples	Valid password, valid token	Roles, scopes, ownership of a record

Common authentication schemes

Scheme	How it works	Testing notes
Basic	Header carries base64(username:password).	Base64 is encoding, not encryption. Only acceptable over HTTPS.
API key	A static key in a header or query string identifies the calling application.	Test missing, invalid and revoked keys. Keys in URLs leak into logs.
Bearer token / JWT	A signed token is sent in Authorization: Bearer ...	Test expiry, tampering, wrong audience and missing scopes.
OAuth 2.0	A token endpoint issues short-lived access tokens, sometimes with refresh tokens.	Most common in modern APIs. Automate token acquisition once and reuse it.
Session cookie	The server issues a session ID in Set-Cookie.	Check Secure, HttpOnly and SameSite flags and logout invalidation.

Anatomy of a JWT

A JSON Web Token has three base64url-encoded parts separated by dots: **header** (algorithm), **payload** (claims such as sub, exp, iat and scope) and **signature**. The payload is *encoded, not encrypted*: anyone can read it, so it must never hold secrets. The signature proves it has not been altered.

```
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJDMTAxIiwiZXhwIjoxNzU4NTQ4MDAwfQ . 3xK9...
        header                        payload (claims)                signature
```

OAuth 2.0 grant types you will automate

Grant	Used for	In test automation
Client credentials	Machine-to-machine calls, no user involved	The easiest and most common choice for API test suites.
Authorization code (with PKCE)	Apps where a person logs in through a browser	Hard to drive headlessly. Use a pre-provisioned test user or a test-only token endpoint.

Grant	Used for	In test automation
Refresh token	Obtaining a new access token without re-login	Useful when the suite must run longer than the access token lifetime.
Password (legacy)	Sending the user's password directly to the token endpoint	Discouraged and dropped from newer OAuth guidance. Only for legacy test environments.

Authentication test checklist

- No credentials returns 401 and does not leak details.
- Invalid, malformed, expired and tampered tokens return 401 with a consistent error body.
- A valid token without the required role or scope returns 403.
- Logout or revocation makes the token unusable.
- Repeated failed logins are rate limited or locked out (429 or an account lock).
- Credentials are never returned in a response, logged or placed in a URL.

Security tests every API tester can run

Risk	Simple test
Broken object-level authorization (BOLA / IDOR)	Call `/accounts/{otherUsersId}` with your own token. Expect 403 or 404, never data.
Excessive data exposure	Check that responses do not contain password hashes, tokens, internal IDs or full card numbers.
Mass assignment	Add unexpected fields such as `"role":"ADMIN"` to a create or update request. They must be ignored or rejected.
Missing rate limiting	Send a burst of requests to a login or search endpoint. Expect 429.
Injection	Send quotes, SQL fragments and script tags in string fields. Expect validation errors, never a 500.
Verbose errors	Trigger errors and confirm no stack traces, SQL or server paths are returned.
Transport security	Confirm HTTP redirects to HTTPS or is refused, and that old TLS versions are disabled.

> **NOTE**
> Broken object-level authorization is the first item in the OWASP API Security Top 10. It is also the easiest serious defect to find with a two-user test.

PART II

Manual API Testing

Learn an API by hand first: Postman workflow, scripting, test-case design and systematic debugging.

CHAPTER 4

Manual API Testing with Postman

Postman is the fastest way to explore an API, reproduce a defect and prototype the checks you will later automate.

A repeatable workflow

1. **Import the contract.** Load the OpenAPI/Swagger file or a cURL command to generate requests.
2. **Create a collection** with one folder per resource (Customers, Payments, Orders).
3. **Create environments** (QA, UAT) holding `baseUrl`, credentials and tokens.
4. **Configure authentication** once at collection level and inherit it in every request.
5. **Send requests** and inspect the status, headers, body and response time.
6. **Add tests** in the Tests tab so each request verifies itself.
7. **Run the collection** with the Collection Runner, then in CI with Newman.

Variable scopes

Postman resolves `{{variable}}` placeholders from the narrowest scope outwards:

Scope	Lifetime	Typical use
Local	One request run	Temporary values inside a script
Data	One iteration of a Runner data file	Data-driven values from CSV or JSON
Environment	Until changed, per environment	baseUrl, tokens, credentials for QA or UAT
Collection	Shared by one collection	Constants shared by all environments
Global	Workspace-wide	Rarely needed. Easy to abuse.

> **WATCH OUT**
> Postman keeps an *initial* value (shared when you sync or export) and a *current* value (local to you). Put secrets in the current value only, or in a vault, and never export an environment file that contains real credentials.

Sending a request, and its cURL twin

Every Postman request can be expressed as cURL, which is ideal for bug reports and for reproducing an issue on any machine.

```
curl -i -X POST "https://qa.example.com/customers" \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer $TOKEN" \
  -d '{"name":"Ajay","mobile":"9876543210"}'
```

Writing tests in the Tests tab

```
pm.test("Status is 201 Created", () => {
    pm.response.to.have.status(201);
});

pm.test("Responds within 3 seconds", () => {
    pm.expect(pm.response.responseTime).to.be.below(3000);
});

pm.test("Content-Type is JSON", () => {
    pm.expect(pm.response.headers.get("Content-Type")).to.include("application/json");
});

const body = pm.response.json();

pm.test("Customer ID and status are correct", () => {
    pm.expect(body.customerId).to.be.a("string").and.not.empty;
    pm.expect(body.status).to.eql("SUCCESS");
});

// Save the ID for the next request (API chaining)
pm.environment.set("customerId", body.customerId);
```

The next request can now use `GET {{baseUrl}}/customers/{{customerId}}`. This is manual API chaining, and it is exactly what you will later code in Rest Assured.

A pre-request script that refreshes the token

Put this at collection level so every request always carries a valid token. The token endpoint and field names are examples.

```
const expiry = Number(pm.environment.get("tokenExpiry") || 0);

if (!pm.environment.get("token") || Date.now() > expiry) {
    pm.sendRequest({
        url: pm.environment.get("baseUrl") + "/auth/token",
        method: "POST",
        header: { "Content-Type": "application/json" },
        body: {
            mode: "raw",
            raw: JSON.stringify({
                clientId: pm.environment.get("clientId"),
                clientSecret: pm.environment.get("clientSecret")
            })
        }
    }, (err, res) => {
        if (err) { console.error(err); return; }
        const json = res.json();
        pm.environment.set("token", json.access_token);
        // renew one minute early
        pm.environment.set("tokenExpiry", Date.now() + (json.expires_in - 60) * 1000);
    });
}
```

Data-driven runs

Create a CSV whose header names match the variables you use in the request body, such as `{"name":"{{name}}","mobile":"{{mobile}}"}`, then select the file in the Collection Runner. Each row becomes one iteration.

```
name,mobile,expectedStatus
Ajay,9876543210,201
Rahul,9876543211,201
,9876543212,400
Meera,abc,400
```

```
const expected = Number(pm.iterationData.get("expectedStatus"));
pm.test("Status matches expectation for this row", () => {
    pm.response.to.have.status(expected);
});
```

Running the collection in CI with Newman

```
npm install -g newman

newman run customer-api.postman_collection.json \
  -e qa.postman_environment.json \
  -d customers.csv \
  --reporters cli,junit \
  --reporter-junit-export results/newman.xml
```

INTERVIEW TIP

Postman is excellent for exploration and light regression. For complex logic, reusable components, database checks and large data sets, move to code (Rest Assured), which is where the rest of this book goes.

CHAPTER 5

Designing API Test Cases

Good API testing is not a list of endpoints. It is a systematic search for the ways a contract can break.

Start from the contract

For each endpoint, write down: the method and path, the required and optional inputs, the allowed values and limits, who may call it, the possible responses, the state it changes and the systems it depends on. Every one of those items produces test cases.

The 12-point coverage checklist

1. **Happy path.** Valid input returns the documented status and body.
2. **Required fields.** Each mandatory field missing, null and empty.
3. **Data types.** A string where a number is expected, and the reverse.
4. **Boundaries.** Minimum, maximum, just inside and just outside every limit.
5. **Formats.** Dates, e-mail, phone, currency and encoding (including Unicode and very long strings).
6. **Authentication.** No token, invalid token, expired token.
7. **Authorization.** Wrong role, wrong owner, wrong tenant.
8. **Headers and content negotiation.** Wrong Content-Type, unsupported Accept.
9. **State and idempotency.** Duplicate submit, update after delete, out-of-order calls.
10. **Collections.** Pagination, sorting, filtering, empty results and very large results.
11. **Error contract.** Consistent error structure, correct codes and no sensitive detail.
12. **Non-functional.** Response time, rate limits, concurrency and schema stability.

Techniques that generate the right cases

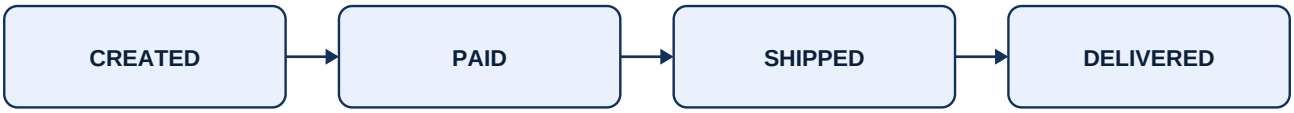
Equivalence partitioning and boundary value analysis

Suppose `amount` must be between 1 and 100000. Group inputs into valid and invalid partitions, then test the edges of each partition.

Input	Partition	Expected
`0`	Below minimum (boundary)	400 - amount must be at least 1
`1`	Minimum valid (boundary)	201
`2`	Just above minimum	201
`99999`	Just below maximum	201
`100000`	Maximum valid (boundary)	201
`100001`	Above maximum (boundary)	400 - amount must not exceed 100000
`-100, 12.999, "ABC", null`	Invalid type, precision or value	400 with a clear message

State transition testing

Many APIs model a lifecycle, for example an order. Test every legal transition and try the illegal ones.



Illegal examples to try: SHIPPED to CREATED, cancel after DELIVERED, pay twice. Each should return a 409 or 422 with an explanation, and leave the order unchanged.

Decision tables and pairwise

When rules combine (customer type, amount, country, payment method), a decision table ensures no combination of business rules is missed. When many optional fields interact, pairwise selection keeps the case count manageable.

Worked example: POST /customers

ID	Scenario	Input	Expected
TC01	Valid customer	name and mobile valid	201, customerId returned, GET returns the same data
TC02	Name missing	no name field	400, message names the field
TC03	Mobile null	"mobile": null	400
TC04	Mobile empty	"mobile": ""	400
TC05	Mobile too short	9 digits	400
TC06	Mobile has letters	"98765abc10"	400
TC07	Duplicate mobile	same mobile as an existing customer	409 (or per business rule)
TC08	Very long name	300+ characters	400, no 500
TC09	Unicode name	"Ajay Kumar (Hindi script)"	201, stored and returned intact
TC10	Wrong Content-Type	text/plain with JSON body	415
TC11	Malformed JSON	trailing comma	400, not 500
TC12	No token	no Authorization header	401
TC13	Read-only role	token without create scope	403
TC14	Extra field	"role": "ADMIN"	Field ignored or 400; role not applied
TC15	Double submit	same request sent twice quickly	One record only, or the documented duplicate behaviour

Testing collections: pagination, sorting, filtering

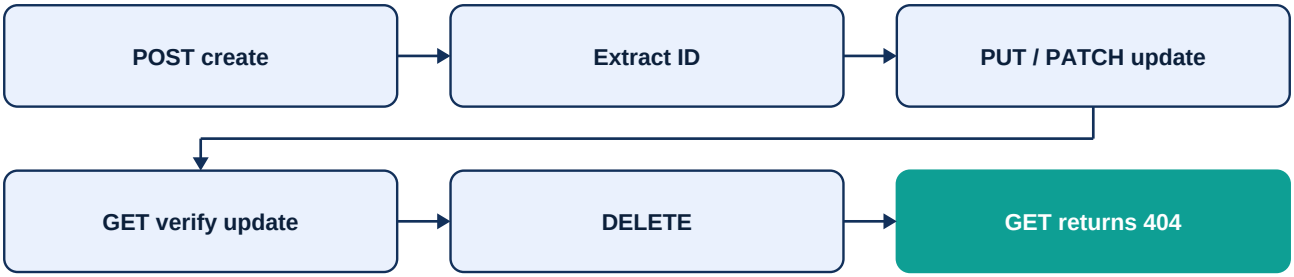
- First page, middle page, last page and a page beyond the end (empty list, not an error).
- Page size of 0, 1, the maximum, and above the maximum.
- Total count and page metadata are consistent with the data.
- Sorting is correct in both directions and stable for equal values.
- Filters combine correctly (AND semantics), and unknown filters are ignored or rejected as documented.

- No record appears twice or goes missing while paging through a changing data set.

Test data strategy

- Generate unique data for every run (UUID or timestamp suffix) so tests never collide.
- Create what you need through the API, and delete it afterwards, so tests are independent.
- Keep a small, read-only seed data set for tests that only read.
- Never depend on data another test created, or on the order tests run in.

An end-to-end lifecycle test



INTERVIEW TIP

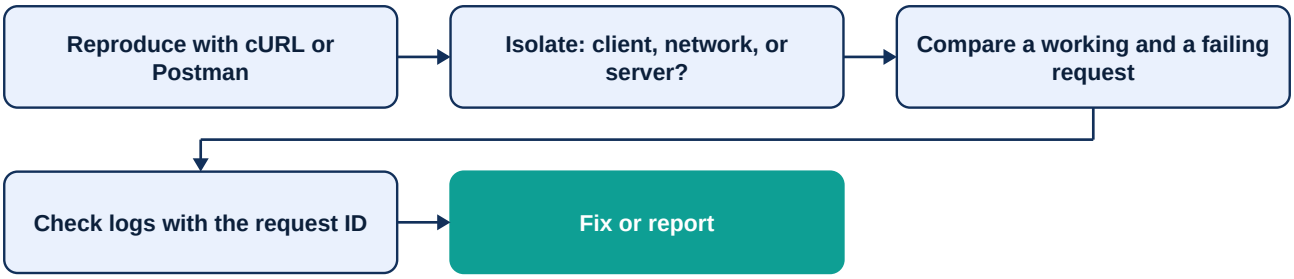
When asked "how would you test this API?", walk through the checklist out loud: happy path, validation, auth, boundaries, state, error contract, then non-functional. It shows structure, not just tool knowledge.

CHAPTER 6

Debugging Failures and Reporting Defects

When a call fails, the fastest testers do not guess. They narrow the problem down in a fixed order.

A systematic approach



Symptom to likely cause

Symptom	Likely causes	What to try
400 Bad Request	Malformed JSON, missing required field, wrong data type	Validate the body in a JSON linter; compare with the schema; look for trailing commas and unescaped quotes.
401 Unauthorized	Missing, expired or invalid token; wrong scheme (Basic vs Bearer)	Decode the JWT and check exp; confirm the header is exactly "Bearer <token>".
403 Forbidden	Valid identity, missing role or scope	Check the user's role, scopes and the resource owner.
404 Not Found	Wrong base URL or path, unresolved path variable, deleted record	Print the final URL; check for double slashes and un-substituted {{placeholders}}.
405 Method Not Allowed	Wrong HTTP method	Check the contract and the Allow response header.
415 Unsupported Media Type	Missing or wrong Content-Type	Send Content-Type: application/json with a JSON body.
422 / 409	Business rule or duplicate	Read the error message; check existing data and state.
429 Too Many Requests	Rate limit exceeded	Honour Retry-After; reduce parallelism in the suite.
500 Internal Server Error	Unhandled server exception	Capture the exact request and request ID, then report it as a defect.
502 / 503 / 504	Gateway, deployment or dependency issue	Check environment health; retry only if the failure is transient.
SSL handshake or PKIX error	Untrusted certificate in a test environment	Install the certificate in the trust store. Relax validation only in non-production.

Symptom	Likely causes	What to try
Timeout	Slow dependency, firewall or VPN	Test from another network; raise the timeout only after understanding the cause.

Reporting a defect developers can act on

- **Title** that states the behaviour: "POST /customers returns 500 when mobile contains letters".
- **Environment and build**, date and time (with time zone) and the request or correlation ID.
- **Exact request**: method, URL, headers (with secrets removed) and body. A cURL command is ideal.
- **Actual vs expected** status code and body, quoting the contract or requirement.
- **Frequency**: always, intermittent, or only for specific data.
- **Evidence**: response body, logs and, where relevant, the database state before and after.

> **INTERVIEW TIP**
> Always capture the correlation or request ID. It lets a developer find your exact call in the logs in seconds, and it turns "it failed once" into a diagnosable defect.

PART III

Automation with Rest Assured

From a first test to 38 interview-ready scenarios: requests, validations, authentication, chaining, payloads and schemas.

CHAPTER 7

Rest Assured Foundations

A Java DSL that makes HTTP tests read like the requirement they verify.

Rest Assured is an open-source Java library for testing REST services. Its BDD-style `given() / when() / then()` syntax keeps each test readable, it uses Hamcrest for assertions and Groovy's GPath for navigating JSON and XML, and it plugs into TestNG or JUnit so your API tests run in the same pipeline as everything else.

Project setup (Maven)

```
<dependencies>
  <dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>rest-assured</artifactId>
    <version>5.5.0</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>json-schema-validator</artifactId>
    <version>5.5.0</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.10.2</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.17.2</version>
  </dependency>
</dependencies>
```

> **NOTE**
> Versions above are illustrative. Check Maven Central for the current stable releases and keep all Rest Assured modules on the same version.

Imports and your first test

```
import static io.restassured.RestAssured.*;
import static org.hamcrest.Matchers.*;
import org.testng.annotations.Test;

public class FirstApiTest {

    @Test
    public void getCustomer_returnsCustomerDetails() {
        given()
            .baseUri("https://qa.example.com")
            .header("Accept", "application/json")
        .when()
            .get("/customers/101")
        .then()
            .statusCode(200)
            .body("name", equalTo("Ajay"))
            .body("status", equalTo("ACTIVE"));
    }
}
```

The three blocks

Block	Role	Typical content
`given()`	Preconditions: everything about the request	baseUri, headers, auth, path/query parameters, body, cookies, spec, logging
`when()`	The action	The HTTP method and path: get(), post(), put(), patch(), delete()
`then()`	Assertions on the response	statusCode(), header(), cookie(), body(), time(), spec(), extract()

Three ways to work with a response

```
// 1. Validate inline
given().when().get("/customers/101").then().statusCode(200).body("id", equalTo(101));

// 2. Extract a single value
String name = given().when().get("/customers/101")
                     .then().statusCode(200)
                     .extract().path("name");

// 3. Keep the whole Response for richer logic
Response response = given().when().get("/customers/101");
int    status = response.statusCode();
String body   = response.asString();
String header = response.header("Content-Type");
long   millis = response.time();
```

Parameters, headers and cookies

```
given()
    .pathParam("id", 101)                  // /customers/{id}
    .queryParam("expand", "orders")        // ?expand=orders
    .queryParams("page", 2, "size", 20)
    .header("X-Correlation-Id", UUID.randomUUID().toString())
    .cookie("region", "IN")
    .formParam("grant_type", "client_credentials")   // form-encoded body
    .multiPart("file", new File("data/statement.pdf")) // file upload
.when()
    .get("/customers/{id}");
```

Logging

```
given().log().all()                    // print the full request
...
.then().log().ifValidationFails()      // print the response only when an assertion fails

// Or once, for the whole suite:
RestAssured.enableLoggingOfRequestAndResponseIfValidationFails();
```

INTERVIEW TIP

Prefer `log().ifValidationFails()` in shared code: failures come with full evidence while passing tests keep the console clean.

Hamcrest matchers you will use daily

Matcher	Meaning	Example
`equalTo(x)`	Equals x	`body("status", equalTo("ACTIVE"))`
`not(m), nullValue()`	Negation, null check	`body("error", nullValue())`
`notNullValue()`	Present and not null	`body("id", notNullValue())`
`containsString / startsWith / endsWith`	Text checks	`body("email", endsWith("@example.com"))`
`greaterThan / lessThan / closeTo`	Numeric comparisons	`body("age", greaterThan(18))`
`hasItem / hasItems`	A collection contains the item(s)	`body("users.id", hasItem(101))`
`hasSize / empty`	Collection size	`body("users", hasSize(2))`
`everyItem(m)`	Every element matches	`body("users.age", everyItem(greaterThan(0)))`
`hasKey / hasEntry`	Map contains key / entry	`body("$", hasKey("id"))`
`allOf / anyOf`	Combine matchers	`body("age", allOf(greaterThan(17), lessThan(66)))`
`matchesPattern`	Regular expression	`body("id", matchesPattern("[a-f0-9-]{36}"))`

Navigating JSON with GPath

Rest Assured evaluates path strings with Groovy's GPath, so you can filter, search and aggregate inside the assertion. Given this response:

```
{
  "total": 2,
  "users": [
    {"id": 101, "name": "Ajay",  "age": 31, "active": true},
    {"id": 102, "name": "Rahul", "age": 17, "active": false}
  ]
}
```

Path expression	Result
`total`	2
`users[0].name`	Ajay
`users.name`	[Ajay, Rahul]
`users.size()`	2
`users.find { it.id == 102 }.name`	Rahul
`users.findAll { it.age >= 18 }.name`	[Ajay]
`users.age.sum()`	48
`users.age.max()`	31
`users.max { it.age }.name`	Ajay

CHAPTER 8

Requests, Responses and Validations

Scenarios 1 to 10: the everyday building blocks of an API automation suite.

Unless stated otherwise, the examples assume a `requestSpec` (base URI, JSON content type, authentication) that Chapter 13 builds properly.

1 How would you automate a POST API?

SHORT ANSWER

Build a request specification with headers and auth, send the JSON payload with given(), then assert the status code, the key response fields and, ideally, that the data was really persisted.

```
@Test
public void createCustomer_returns201() {
    Map<String, Object> payload = Map.of("name", "Ajay", "mobile", "9876543210");

    given()
        .spec(requestSpec)
        .body(payload)
    .when()
        .post("/customers")
    .then()
        .statusCode(201)
        .header("Location", containsString("/customers/"))
        .body("status", equalTo("SUCCESS"))
        .body("customerId", notNullValue());
}
```

WATCH OUT

When you pass a Map or POJO to `body()`, Rest Assured needs a content type (and Jackson or Gson on the classpath) to know how to serialise it. Set `ContentType.JSON` in the spec.

INTERVIEW TIP

A 201 only proves the API said yes. Follow with a GET, as shown in Scenario 19, to prove the data was stored.

2 You have 10 test cases with different payloads. How do you manage them?

SHORT ANSWER

Keep the test logic in one method and feed the data from a TestNG DataProvider backed by an external file (JSON, CSV or Excel), never hard-coded in the test.

```
@DataProvider(name = "customers")
public Object[][] customers() {
    return new Object[][] {
        // name,    mobile,        expected status
        {"Ajay",    "9876543210",  201},
        {"Rahul",   "9876543211",  201},
        {"",        "9876543212",  400},
        {"Meera",   "abc",         400}
    };
}

@Test(dataProvider = "customers")
public void createCustomer(String name, String mobile, int expectedStatus) {
    Map<String, Object> body = new HashMap<>();
    body.put("name", name);
    body.put("mobile", mobile);

    given().spec(requestSpec).body(body)
    .when().post("/customers")
    .then().statusCode(expectedStatus);
}
```

For larger data sets, read the rows from a file so testers can add cases without touching Java:

```
@DataProvider(name = "fromJson")
public Object[][] fromJson() throws IOException {
    List<Map<String, Object>> rows = new ObjectMapper().readValue(
        new File("src/test/resources/testdata/customers.json"),
        new TypeReference<List<Map<String, Object>>>() {});
    return rows.stream().map(r -> new Object[]{r}).toArray(Object[][]::new);
}
```

INTERVIEW TIP

Add `parallel = true` to the `@DataProvider` to run rows concurrently, but only after making sure the rows do not share mutable data.

3 How do you send a dynamic JSON request body?

SHORT ANSWER

Build it from a Map, a POJO serialised by Jackson, a JSON template file, or a text block. In a real framework I prefer POJOs.

Option	Best for	Trade-off
Map<String,Object>	Quick tests, negative cases, null values	No compile-time safety on field names.
POJO + Jackson	Framework-grade tests, nested structures	More classes to maintain, but refactor-safe and reusable.
JSON file / template	Large, mostly static payloads	Needs a small utility to override fields.
Text block (Java 15+)	Readable inline JSON, and invalid JSON for negative tests	Values are strings, so easy to break the syntax.

```
// Map
Map<String, Object> payload = new HashMap<>();
payload.put("name", "Ajay");
payload.put("amount", 500);
given().spec(requestSpec).body(payload).when().post("/payment");

// File
given().spec(requestSpec)
       .body(new File("src/test/resources/payloads/customer.json"))
       .when().post("/customers");

// Text block with a dynamic value
String json = """
    {
      "name": "%s",
      "mobile": "%s"
    }
    """.formatted("Ajay", "9876543210");
given().spec(requestSpec).body(json).when().post("/customers");
```

4 How do you validate a specific key and value in the response?

SHORT ANSWER

Use the JSON path in `body()` with a Hamcrest matcher, or extract the value and use TestNG assertions.

```
given().spec(requestSpec)
.when().get("/customer/101")
.then()
    .statusCode(200)
    .body("name",   equalTo("Ajay"))
    .body("status", equalTo("ACTIVE"))
    .body("age",    greaterThan(18));
```

5 How do you validate multiple JSON fields?

SHORT ANSWER

Chain several body() assertions, pass key-matcher pairs in one call, or extract the response and use SoftAssert so every mismatch is reported in one run.

```
// Pairs in a single call
.then().statusCode(200)
       .body("id", equalTo(101),
             "name", equalTo("Ajay"),
             "status", equalTo("ACTIVE"));

// SoftAssert: report all failures, not just the first
Response response = given().spec(requestSpec).when().get("/customer/101");
JsonPath json = response.jsonPath();

SoftAssert soft = new SoftAssert();
soft.assertEquals(response.statusCode(), 200, "status code");
soft.assertEquals(json.getInt("id"), 101, "id");
soft.assertEquals(json.getString("name"), "Ajay", "name");
soft.assertEquals(json.getString("status"), "ACTIVE", "status");
soft.assertAll();   // do not forget this line
```

WATCH OUT

Forgetting `assertAll()` makes a SoftAssert test pass even when assertions failed.

6 How do you extract a transactionId and use it in another API?

SHORT ANSWER

This is API chaining: extract the value from the first response and pass it as a path parameter, query parameter or body field of the next request.

```
String transactionId =
    given().spec(requestSpec)
           .body(payload)
    .when()
        .post("/transaction")
    .then()
        .statusCode(201)
        .extract().path("transactionId");

given().spec(requestSpec)
       .pathParam("id", transactionId)
.when()
    .get("/transaction/{id}")
.then()
    .statusCode(200)
    .body("transactionId", equalTo(transactionId));
```

7 How do you validate that a JSON array contains a particular object?

SHORT ANSWER

Use `hasItem` on a projected path for simple checks, and `find { ... }` or `hasItem(allOf(hasEntry...))` when several fields must belong to the *same* object.

```
// Response: {"users":[{"id":101,"name":"Ajay"},{"id":102,"name":"Rahul"}]}

.then()
    .body("users.id",   hasItem(101))
    .body("users.name", hasItems("Ajay", "Rahul"))

    // same object: id 101 must be named Ajay
    .body("users.find { it.id == 101 }.name", equalTo("Ajay"))
    .body("users", hasItem(allOf(hasEntry("id", 101), hasEntry("name", "Ajay"))));
```

WATCH OUT

`hasItem(101)` on `users.id` and `hasItem("Ajay")` on `users.name` can both pass even if Ajay belongs to a different ID. Assert on the object when the pairing matters.

8 How do you validate response time?

SHORT ANSWER

Use `time(lessThan(...))` against an agreed SLA, with headroom for CI variance. Use a load tool such as JMeter for real performance testing.

```
.then()
    .statusCode(200)
    .time(lessThan(3000L));                          // milliseconds

.then().time(lessThan(3L), TimeUnit.SECONDS);        // explicit unit

// or read it and decide yourself
long millis = response.time();
```

INTERVIEW TIP

A single-request time check is a smoke-level guard, not a performance test. Shared CI runners add noise, so choose a realistic SLA rather than a razor-thin one, and track trends rather than single readings.

9 How do you validate response headers?

SHORT ANSWER

Use `header(name, matcher)` or `headers(...)` in the then() block.

```
.then()
    .header("Content-Type", containsString("application/json"))
    .header("Cache-Control", notNullValue())
    .headers("X-Content-Type-Options", "nosniff",
             "Strict-Transport-Security", containsString("max-age"));
```

Beyond correctness, header checks are cheap security assertions: content-type sniffing protection, HSTS, and the absence of headers that reveal server internals.

10 How do you validate cookies?

SHORT ANSWER

Assert the cookie's presence and value with `cookie()`, or read it from the Response and check its attributes.

```
.then().cookie("sessionId", notNullValue());

Response response = given().spec(requestSpec).when().post("/login");
String sessionId = response.getCookie("sessionId");

Cookie detailed = response.getDetailedCookie("sessionId");
assertTrue(detailed.isSecured(),  "Secure flag");
assertTrue(detailed.isHttpOnly(), "HttpOnly flag");
```

CHAPTER 9

Authentication in Automation

Scenarios 11 to 18: token handling that survives expiry, environments and parallel execution.

11 How do you pass a Bearer token?

SHORT ANSWER

Set the Authorization header to "Bearer <token>", or use `auth().oauth2(token)`, which does the same thing.

```
given().header("Authorization", "Bearer " + token)
.when().get("/accounts")
.then().statusCode(200);

// equivalent, and clearer
given().auth().oauth2(token)
.when().get("/accounts")
.then().statusCode(200);
```

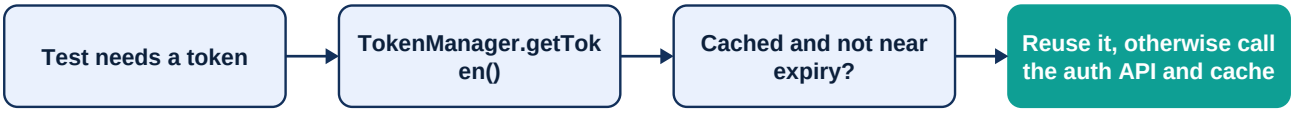
INTERVIEW TIP

Never hard-code a token in a test. It expires, it leaks through source control, and it ties the test to one environment. Obtain it through a token manager (next scenario).

12 The token expires after 30 minutes. What will you do?

SHORT ANSWER

Centralise token handling in a TokenManager that caches the token with its expiry time and refreshes it only when it is missing or about to expire.




```
public final class TokenManager {

    private static String token;
    private static Instant expiresAt = Instant.EPOCH;
    private static final Duration SAFETY_BUFFER = Duration.ofMinutes(2);

    private TokenManager() { }

    public static synchronized String getToken() {
        if (token == null || Instant.now().isAfter(expiresAt.minus(SAFETY_BUFFER))) {
            refresh();
        }
        return token;
    }

    private static void refresh() {
```

```
        Response res = given()
            .baseUri(ConfigReader.get("authUrl"))
            .contentType(ContentType.URLENCODED)
            .formParam("grant_type", "client_credentials")
            .formParam("client_id", ConfigReader.get("clientId"))
            .formParam("client_secret", ConfigReader.get("clientSecret"))
        .when()
            .post("/oauth/token")
        .then()
            .statusCode(200)
            .extract().response();

        token = res.path("access_token");
        int ttlSeconds = res.path("expires_in");
        expiresAt = Instant.now().plusSeconds(ttlSeconds);
    }
}
```

- **Buffer:** refresh a couple of minutes early so a token never expires mid-request.
- **synchronized:** when tests run in parallel, only one thread refreshes and the rest reuse the result.
- **Fallback:** as a safety net, a Rest Assured filter can refresh once and retry when it sees a 401.

INTERVIEW TIP

The senior-level point: do not generate a token before every API call. Generate once, reuse until near expiry, and make the logic thread-safe.

13 How do you implement OAuth 2.0?

SHORT ANSWER

Call the token endpoint (usually client credentials for automation), extract the access_token, and send it with `auth().oauth2()`. Use a refresh token when the suite outlives the access token.

```
// Client credentials: no user interaction, ideal for suites
String accessToken =
    given()
        .baseUri(authUrl)
        .contentType(ContentType.URLENCODED)
        .formParam("grant_type", "client_credentials")
        .formParam("client_id", clientId)
        .formParam("client_secret", clientSecret)
        .formParam("scope", "customers.read customers.write")
    .when().post("/oauth/token")
    .then().statusCode(200)
           .body("token_type", equalToIgnoringCase("bearer"))
           .extract().path("access_token");

given().auth().oauth2(accessToken)
.when().get("/profile")
.then().statusCode(200);
```

With the authorization-code flow, a person normally logs in through a browser, which is awkward to automate. Ask for a dedicated test client with client credentials, a service account, or a pre-issued refresh token, and exchange that for access tokens with `grant_type=refresh_token`.

14 How do you handle Basic Authentication?

SHORT ANSWER

Use `auth().preemptive().basic(user, pass)`, and read the credentials from configuration or CI secrets.

```
given()
    .auth().preemptive().basic(ConfigReader.get("username"), System.getenv("API_PASSWORD"))
.when()
    .get("/users")
.then()
    .statusCode(200);
```

WATCH OUT

Plain `auth().basic()` waits for a 401 challenge before sending credentials. Many APIs never issue that challenge and simply return 401, so preemptive is the safer choice.

15 How do you use different credentials for different environments?

SHORT ANSWER

Load environment-specific properties chosen by a command-line parameter, and keep secrets out of the repository by reading them from environment variables or the CI credential store.

```
src/test/resources/
    config-qa.properties        baseUrl=https://qa.example.com
    config-uat.properties       baseUrl=https://uat.example.com
```

```
public final class ConfigReader {

    private static final Properties PROPS = new Properties();

    static {
        String env = System.getProperty("env", "qa").toLowerCase();
        try (InputStream in = ConfigReader.class.getClassLoader()
                .getResourceAsStream("config-" + env + ".properties")) {
            if (in == null) {
                throw new IllegalStateException("No config file for env: " + env);
            }
            PROPS.load(in);
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
    }

    public static String get(String key) {
        String override = System.getProperty(key);            // -Dkey=value wins
        if (override != null) return override;
        String secret = System.getenv(key.toUpperCase());     // CI secrets
        return secret != null ? secret : PROPS.getProperty(key);
    }
}
```

```
mvn clean test -Denv=uat
```

16 A token from API-1 is needed by APIs 2, 3 and 4. How do you design it?

SHORT ANSWER

Keep authentication in one place. A request-spec factory asks the TokenManager for a token whenever it builds a spec, so no test ever handles tokens itself.

```
public final class RequestSpecFactory {

    public static RequestSpecification authenticated() {
        return new RequestSpecBuilder()
            .setBaseUri(ConfigReader.get("baseUrl"))
            .setContentType(ContentType.JSON)
            .addHeader("Authorization", "Bearer " + TokenManager.getToken())
            .build();
    }
}

// in any test
given().spec(RequestSpecFactory.authenticated()).when().get("/orders");
```

WATCH OUT

Do not build the spec once in a static field at class-load time. The token baked into it will go stale during a long run. Building a spec per call is cheap.

17 How do you test an invalid or expired token?

SHORT ANSWER

Send deliberately bad tokens and assert 401 plus a consistent error body. Cover each way a token can be wrong, not just one.

Case	How to produce it	Expected
No token	Omit the Authorization header	401
Garbage token	"invalid-token"	401
Expired token	A saved expired JWT, or a test client with a 1-second lifetime	401
Tampered token	Alter one character of the payload or signature	401
Wrong scheme	"Basic <token>" instead of Bearer	401
Valid token, missing scope	A token from a low-privilege client	403

```
@DataProvider(name = "badTokens")
public Object[][] badTokens() {
    return new Object[][] {
        {"invalid-token"},
        {"eyJhbGciOiJIUzI1NiJ9.e30.tampered"},
        {EXPIRED_JWT}
    };
}

@Test(dataProvider = "badTokens")
public void badToken_isRejected(String token) {
    given().baseUri(baseUrl).auth().oauth2(token)
    .when().get("/accounts")
    .then().statusCode(401)
           .body("errorCode", equalTo("INVALID_TOKEN"));
}
```

18 How do you verify that one user cannot access another user's data?

SHORT ANSWER

Test authorization boundaries with two identities: each user gets 200 on their own resources and 403 (or 404) on the other's.

Caller	Resource	Expected
User A	User A's account	200
User A	User B's account	403 (or 404 if the API hides existence)
Admin	User B's account	200 (if admins are allowed)
No token	Any account	401

```
@Test
public void userCannotReadAnotherUsersAccount() {
    given().auth().oauth2(userAToken)
    .when().get("/users/{id}/account", userBId)
    .then().statusCode(anyOf(is(403), is(404)))
           .body("$", not(hasKey("balance")));      // and no data leaked either way
}
```

NOTE

This is a broken object-level authorization (BOLA / IDOR) test, the top item in the OWASP API Security list.

CHAPTER 10

Dynamic Data and API Chaining

Scenarios 19 to 24: real workflows depend on values that only exist at runtime.

19 Create a customer, capture the customerId, then fetch that customer.

SHORT ANSWER

POST, extract the ID, GET by that ID, and compare the fetched data with what you sent.

```
@Test
public void createdCustomerCanBeFetched() {
    Map<String, Object> payload = PayloadFactory.newCustomer();   // unique data

    String customerId =
        given().spec(requestSpec).body(payload)
        .when().post("/customers")
        .then().statusCode(201)
               .extract().path("customerId");

    given().spec(requestSpec).pathParam("id", customerId)
    .when().get("/customers/{id}")
    .then().statusCode(200)
           .body("customerId", equalTo(customerId))
           .body("name",   equalTo(payload.get("name")))
           .body("mobile", equalTo(payload.get("mobile")));
}
```

20 How do you store a dynamic orderId and reuse it?

SHORT ANSWER

Read it from the Response object, keep it in a local variable inside the test, and pass it to the next call.

```
Response created = given().spec(requestSpec).body(payload)
                          .when().post("/orders");

String orderId = created.jsonPath().getString("orderId");

given().spec(requestSpec).pathParam("orderId", orderId)
.when().get("/orders/{orderId}")
.then().statusCode(200);
```

WATCH OUT

Avoid storing such values in `static` fields shared across tests. They cause hidden ordering dependencies and break under parallel execution. Prefer one test per complete flow, with the value in a local variable.

21 How do you extract multiple values from a response?

SHORT ANSWER

Use jsonPath() typed getters, or deserialise the whole response into a POJO with `as(Class)`.

```
Response response = given().spec(requestSpec).when().get("/customer/101");
JsonPath json = response.jsonPath();

int          id      = json.getInt("id");
String       name    = json.getString("name");
List<String> phones  = json.getList("phones");
Map<String, ?> addr  = json.getMap("address");

// or the whole thing as an object
Customer customer = response.as(Customer.class);
assertEquals(customer.getName(), "Ajay");
```

22 How do you feed a response value into the next request's payload?

SHORT ANSWER

Extract the value, then set it on the next payload, preferably through a POJO so field names are compile-checked.

```
String customerId = createCustomer().jsonPath().getString("customerId");

PaymentRequest payment = new PaymentRequest();
payment.setCustomerId(customerId);
payment.setAmount(1000);
payment.setCurrency("INR");

given().spec(requestSpec).body(payment)
.when().post("/payment")
.then().statusCode(201)
       .body("customerId", equalTo(customerId));
```

23 The API returns a random UUID. How do you validate it?

SHORT ANSWER

Never compare it with a hard-coded value. Assert that it is present and well-formed, and use it for the follow-up calls.

```
// Pattern
.then().body("id", matchesPattern(
    "[0-9a-fA-F]{8}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{4}-[0-9a-fA-F]{12}"));

// Or let Java validate the format: UUID.fromString throws if it is malformed
String id = response.jsonPath().getString("id");
assertNotNull(UUID.fromString(id));
```

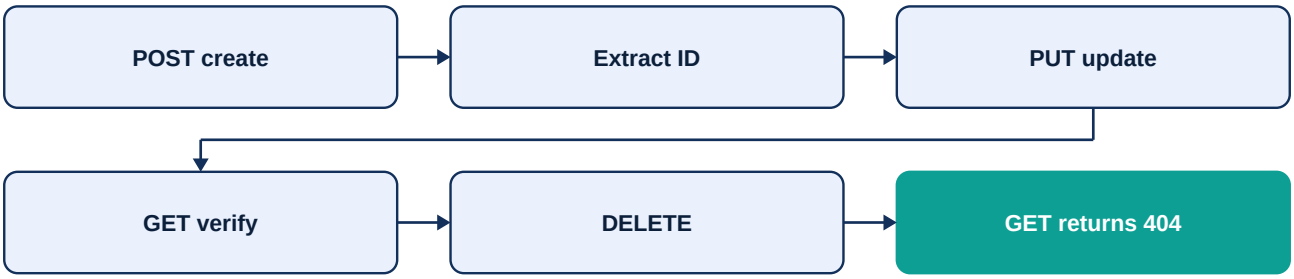
The same idea applies to timestamps: parse them and check they fall within a sensible window instead of comparing exact values.

```
Instant createdAt = Instant.parse(response.jsonPath().getString("createdAt"));
assertTrue(createdAt.isAfter(Instant.now().minusSeconds(60)), "created within the last minute");
```

24 How do you automate a Create, Update, Get, Delete workflow?

SHORT ANSWER

Chain the calls in one test, assert the status and the important business fields at every step, and always clean up, even when an assertion fails.




```
@Test
public void customerLifecycle() {
    String id = null;
    try {
        id = given().spec(requestSpec).body(PayloadFactory.newCustomer())
             .when().post("/customers")
             .then().statusCode(201).extract().path("customerId");

        given().spec(requestSpec).pathParam("id", id)
               .body(Map.of("name", "Ajay Updated"))
        .when().put("/customers/{id}")
        .then().statusCode(200);

        given().spec(requestSpec).pathParam("id", id)
        .when().get("/customers/{id}")
        .then().statusCode(200).body("name", equalTo("Ajay Updated"));

        given().spec(requestSpec).pathParam("id", id)
        .when().delete("/customers/{id}")
        .then().statusCode(anyOf(is(200), is(204)));

        given().spec(requestSpec).pathParam("id", id)
        .when().get("/customers/{id}")
        .then().statusCode(404);
        id = null;                                  // already deleted
    } finally {
        if (id != null) {                           // clean up after a failure
            given().spec(requestSpec).pathParam("id", id)
                   .when().delete("/customers/{id}");
        }
    }
}
```

CHAPTER 11

Payload Design and Validation

Scenarios 25 to 32: building, varying and validating request bodies without duplication.

25 How do you validate the request amount before sending it?

SHORT ANSWER

Put the rule in a payload builder that fails fast on invalid test input, and keep API-level rejection of bad amounts as separate negative tests.

```
public PaymentRequest build() {
    if (amount <= 0) {
        throw new IllegalArgumentException("Test bug: amount must be positive, was " + amount);
    }
    return new PaymentRequest(customerId, amount, currency);
}
```

Notice the two different goals. A builder guard catches mistakes in your *test data*. To verify the *API* rejects a negative amount, deliberately bypass the guard (for example with a raw Map) and assert the 400 response.

26 How do you validate the request payload key and value, not the response?

SHORT ANSWER

Assert on the payload object you built, or read back what the request specification is actually about to send. Request checks and response checks are separate concerns.

```
// 1. On your own object (payload builder unit tests)
PaymentRequest request = new PaymentRequest();
request.setAmount(500);
request.setCurrency("INR");
assertEquals(request.getAmount(), 500);
assertEquals(request.getCurrency(), "INR");

// 2. On what the specification holds
RequestSpecification spec = given().contentType(ContentType.JSON).body(request);
QueryableRequestSpecification query = SpecificationQuerier.query(spec);
Object sentBody = query.getBody();            // the object handed to body()
assertNotNull(sentBody);
```

INTERVIEW TIP

To prove what really went over the wire, capture it with a request filter (see Scenario 46) or log().body(). The true test of a request is what the server did with it, so follow up with a GET.

27 How do you handle a nested JSON payload?

SHORT ANSWER

Model it with nested POJOs and let Jackson serialise it. It is type-safe, reusable and easy to refactor.

```
@JsonInclude(JsonInclude.Include.NON_NULL)
public class Customer {
    private String name;
    private Address address;
    // getters and setters (or Lombok @Data / @Builder)
}

public class Address {
    private String city;
    private String country;
    // getters and setters
}

Customer customer = new Customer();
customer.setName("Ajay");
customer.setAddress(new Address("Delhi", "IN"));

given().spec(requestSpec).body(customer)
.when().post("/customers")
.then().statusCode(201);
```

For responses, add `@JsonIgnoreProperties(ignoreUnknown = true)` to POJOs so a harmless new field in the API does not break deserialisation.

28 The payload has 50 fields and only 5 change between environments. What do you do?

SHORT ANSWER

Keep one base JSON template and override only the fields that vary, taken from configuration. Never maintain 50-field copies per environment.

```
public final class PayloadFactory {
    private static final ObjectMapper MAPPER = new ObjectMapper();

    public static ObjectNode customer() {
        try {
            ObjectNode payload = (ObjectNode) MAPPER.readTree(
                new File("src/test/resources/payloads/base-customer.json"));

            payload.put("country",  ConfigReader.get("country"));
            payload.put("currency", ConfigReader.get("currency"));
            payload.put("mobile",   RandomData.mobile());                // unique per run
            payload.withObject("/address").put("city", ConfigReader.get("city"));
            return payload;
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
    }
}

given().spec(requestSpec).body(PayloadFactory.customer().toString())
       .when().post("/customers");
```

29 How do you modify one specific JSON field dynamically?

SHORT ANSWER

Change it on a Map, POJO or Jackson tree before sending. Rest Assured's own JsonPath is read-only.

Approach	Example
POJO / Map setter	`customer.setName("Rahul"); / map.put("name", "Rahul");`
Jackson tree	`node.withObject("/customer").put("name", "Rahul");`
Jayway JsonPath (extra library)	`JsonPath.parse(json).set("$.customer.name", "Rahul").jsonString();`

```
JsonNode root = new ObjectMapper().readTree(jsonPayload);
((ObjectNode) root.at("/customer")).put("name", "Rahul");
String updatedPayload = root.toPrettyString();
```

30 How do you validate mandatory fields?

SHORT ANSWER

Start from one fully valid payload, remove one field at a time, and assert a 400 that names the field.

```
@DataProvider(name = "requiredFields")
public Object[][] requiredFields() {
    return new Object[][] {{"name"}, {"mobile"}, {"email"}};
}

@Test(dataProvider = "requiredFields")
public void missingRequiredField_isRejected(String field) {
    ObjectNode payload = PayloadFactory.customer();
    payload.remove(field);

    given().spec(requestSpec).body(payload.toString())
    .when().post("/customers")
    .then().statusCode(400)
           .body("message", containsStringIgnoringCase(field));
}
```

31 How do you test empty, null, missing and invalid fields?

SHORT ANSWER

Treat each as its own scenario. They often take different code paths and produce different errors.

Variant	Example for amount	Typical expectation
Missing	`field absent`	400: amount is required
Null	`"amount": null`	400
Empty	`"amount": ""`	400
Invalid value	`"amount": -100`	400 or 422
Wrong type	`"amount": "ABC"`	400
Boundary	`0, 1, max, max + 1`	Per business rules

```
@DataProvider(name = "badAmounts")
public Object[][] badAmounts() {
    return new Object[][] {{null}, {""}, {-100}, {"ABC"}, {0}};
}

@Test(dataProvider = "badAmounts")
public void invalidAmount_isRejected(Object amount) {
    Map<String, Object> body = new HashMap<>();      // Map.of() rejects null values
    body.put("customerId", "C101");
    body.put("amount", amount);

    given().spec(requestSpec).body(body)
    .when().post("/payment")
    .then().statusCode(400);
}
```

WATCH OUT

`Map.of(...)` throws a NullPointerException on null values. Use a `HashMap` when a test needs to send a JSON null.

32 How do you generate a payload using a POJO?

SHORT ANSWER

Create a POJO, set the fields (preferably through a builder), and pass it to body() with a JSON content type.

```
PaymentRequest request = PaymentRequest.builder()
        .customerId("C101")
        .amount(500)
        .currency("INR")
        .build();

PaymentResponse response =
    given().spec(requestSpec).body(request)
    .when().post("/payment")
    .then().statusCode(201)
           .extract().as(PaymentResponse.class);

assertEquals(response.getAmount(), 500);
assertEquals(response.getStatus(), "SUCCESS");
```

INTERVIEW TIP

Use a *test data builder* with sensible valid defaults. Each test then overrides only the field it is about, which makes its intent obvious.

CHAPTER 12

JSON and Schema Validation

Scenarios 33 to 38: proving the shape, types and safety of responses, not just a few values.

33 How do you do JSON schema validation?

SHORT ANSWER

Add the json-schema-validator module and match the response against a schema file with `matchesJsonSchemaInClasspath`.

```
{
  "$schema": "http://json-schema.org/draft-07/schema#",
  "type": "object",
  "required": ["id", "name", "status"],
  "properties": {
    "id":     { "type": "integer" },
    "name":   { "type": "string", "minLength": 1 },
    "status": { "type": "string", "enum": ["ACTIVE", "INACTIVE"] },
    "email":  { "type": ["string", "null"] }
  },
  "additionalProperties": false
}
```

```
import static io.restassured.module.jsv.JsonSchemaValidator.matchesJsonSchemaInClasspath;

given().spec(requestSpec)
.when().get("/customers/101")
.then().statusCode(200)
       .body(matchesJsonSchemaInClasspath("schemas/customer-schema.json"));
```

- It verifies required fields, types, allowed values, nesting and array structure.
- It does *not* verify business correctness. Combine it with value assertions.
- Setting `additionalProperties` to false catches unexpected new fields, which is useful for contract testing but can be noisy. Choose deliberately.

34 How do you handle dynamic values such as timestamps and UUIDs?

SHORT ANSWER

Assert their *properties* (present, well-formed, within a range) instead of their value, and strip them out before comparing whole objects.

```
.then()
    .body("id", matchesPattern("[0-9a-fA-F-]{36}"))
    .body("createdAt", notNullValue());

// Compare everything else against an expected file
ObjectNode actual = (ObjectNode) MAPPER.readTree(response.asString());
actual.remove(List.of("id", "createdAt", "updatedAt"));

JsonNode expected = MAPPER.readTree(new File("src/test/resources/expected/customer.json"));
assertEquals(actual, expected);
```

35 How do you validate JSON data types?

SHORT ANSWER

Prefer schema validation. For quick checks, assert the Java type with `instanceOf`.

```
.then()
    .body("id",     instanceOf(Integer.class))
    .body("name",   instanceOf(String.class))
    .body("active", instanceOf(Boolean.class))
    .body("phones", instanceOf(List.class));
```

WATCH OUT

Rest Assured parses JSON decimals as `float` by default, so `body("price", equalTo(19.99))` can fail. Compare with `19.99f`, or switch the number type once for the whole suite.

```
RestAssured.config = RestAssuredConfig.config().jsonConfig(
    JsonConfig.jsonConfig().numberReturnType(JsonPathConfig.NumberReturnType.BIG_DECIMAL));

.then().body("price", equalTo(new BigDecimal("19.99")));
```

36 How do you validate nested JSON?

SHORT ANSWER

Use dot notation to walk the structure, brackets for array positions, and GPath filters to find the right element.

```
// {"customer":{"address":{"city":"Delhi"},"orders":[{"id":1,"items":[{"price":10}]}]}}

.then()
    .body("customer.address.city",           equalTo("Delhi"))
    .body("customer.orders[0].id",           equalTo(1))
    .body("customer.orders[0].items[0].price", equalTo(10))
    .body("customer.orders.size()",           equalTo(1));
```

37 The response contains an array of 100 objects. How do you validate it?

SHORT ANSWER

Check the size, check that every element satisfies the rule, then extract the list for anything Hamcrest cannot express.

```
.then()
    .body("users",         hasSize(100))
    .body("users.id",      everyItem(notNullValue()))
    .body("users.status",  everyItem(anyOf(equalTo("ACTIVE"), equalTo("INACTIVE"))))
    .body("users.find { it.id == 101 }.name", equalTo("Ajay"));

// Extract for richer checks
List<Integer> ids = response.jsonPath().getList("users.id");
assertEquals(new HashSet<>(ids).size(), ids.size(), "IDs must be unique");

List<String> names = response.jsonPath().getList("users.name");
List<String> sorted = new ArrayList<>(names);
Collections.sort(sorted);
assertEquals(names, sorted, "names must be sorted A-Z");
```

38 How do you verify the response does not contain a sensitive key?

SHORT ANSWER

Assert the key is absent with `not(hasKey(...))` on the JSON object, and check the nested objects too.

```
.then()
    .body("$",        not(hasKey("password")))
    .body("customer", not(hasKey("ssn")));

// a whole list of forbidden keys
@DataProvider(name = "forbiddenKeys")
public Object[][] forbiddenKeys() {
    return new Object[][] {{"password"}, {"passwordHash"}, {"token"}, {"cardNumber"}};
}

@Test(dataProvider = "forbiddenKeys")
public void sensitiveKeyIsNeverReturned(String key) {
    given().spec(requestSpec).when().get("/customers/101")
    .then().body("$", not(hasKey(key)));
}
```

INTERVIEW TIP

A crude `asString().contains("password")` check also fires on harmless values and misses nested keys. The JSON-aware matcher is precise. Mention that this is an *excessive data exposure* test.

PART IV

Framework, CI/CD and Senior Scenarios

How to turn individual tests into a maintainable framework, run it in a pipeline, and answer the questions senior interviewers really ask.

CHAPTER 13

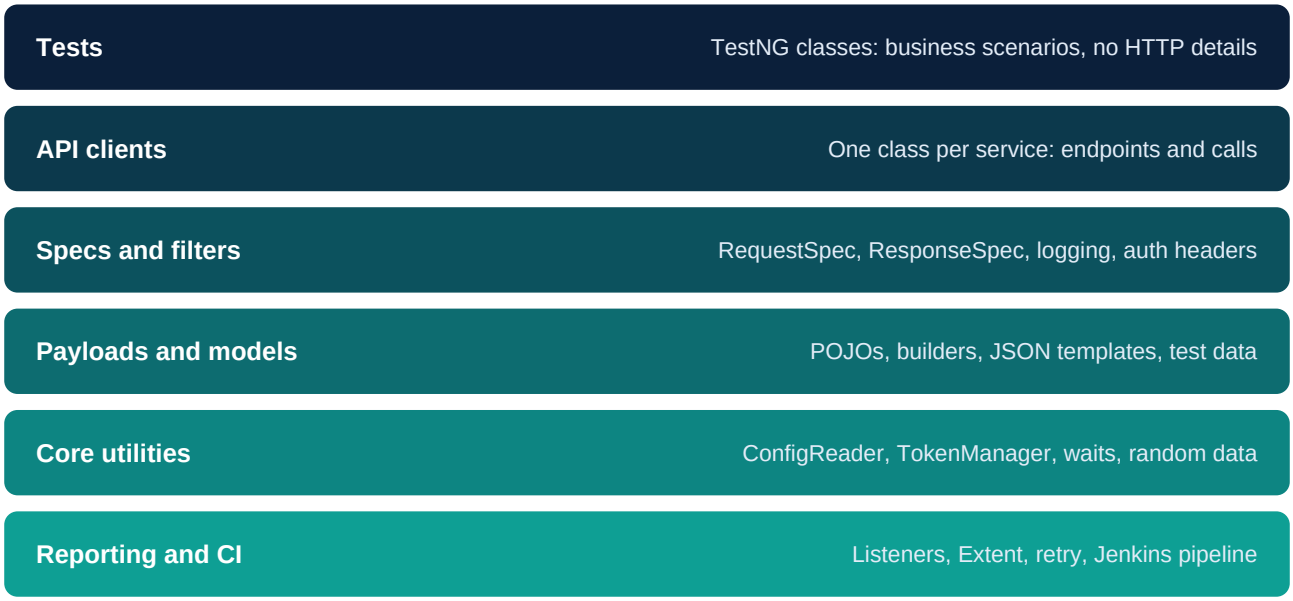
Framework Architecture

Scenarios 39 to 43: the design that lets 500 APIs stay maintainable.

39 You have 500 APIs to automate. How would you design the framework?

SHORT ANSWER

A layered architecture in which tests describe business behaviour, API clients own the HTTP details, and shared concerns (config, auth, specs, data, reporting) live in one place each.




```
api-automation/
|-- pom.xml
|-- testng.xml
`-- src/
    |-- main/java/com/example/api/
    |   |-- clients/      CustomerApi, PaymentApi, AuthApi
    |   |-- config/       ConfigReader
    |   |-- auth/         TokenManager
    |   |-- specs/        RequestSpecFactory, ResponseSpecs
    |   |-- models/       Customer, Address, PaymentRequest
    |   |-- payloads/     PayloadFactory, RandomData
    |   |-- filters/      ApiLogFilter
    |   `-- utils/        JsonUtil, WaitUtil
    `-- test/
        |-- java/com/example/api/
        |   |-- base/         BaseTest
        |   |-- listeners/    TestListener, RetryAnalyzer
        |   `-- tests/        customer/, payment/, security/
        `-- resources/
            |-- config-qa.properties, config-uat.properties
            |-- payloads/     base-customer.json
            |-- schemas/      customer-schema.json
            `-- testdata/     customers.json
```

Component	Responsibility
BaseTest	Suite-level setup: logging, timeouts, environment safety checks.
RequestSpecFactory / ResponseSpecs	Reusable request and response configuration.
TokenManager, ConfigReader	Authentication and environment configuration in one place each.
API clients (service objects)	Wrap each endpoint so tests never repeat paths and headers.
Models and PayloadFactory	Type-safe request and response objects, plus valid test data with unique values.
ApiLogFilter, listeners	Evidence capture and report integration.
RetryAnalyzer	Controlled re-execution of transient failures.

An API client keeps the endpoint knowledge out of the tests. It returns the raw `Response` so tests can still assert on anything:

```
public class CustomerApi {

    private static final String CUSTOMERS = "/customers";

    public Response create(Object body) {
        return given().spec(RequestSpecFactory.authenticated())
                      .body(body)
               .when().post(CUSTOMERS);
    }

    public Response getById(String id) {
        return given().spec(RequestSpecFactory.authenticated())
                      .pathParam("id", id)
               .when().get(CUSTOMERS + "/{id}");
    }

    public Response delete(String id) {
        return given().spec(RequestSpecFactory.authenticated())
                      .pathParam("id", id)
               .when().delete(CUSTOMERS + "/{id}");
    }
}
```

```
public class CustomerTests extends BaseTest {

    private final CustomerApi customerApi = new CustomerApi();

    @Test
    public void createdCustomerCanBeFetched() {
        Response created = customerApi.create(PayloadFactory.customer().toString());
        assertEquals(created.statusCode(), 201);

        String id = created.jsonPath().getString("customerId");
        Response fetched = customerApi.getById(id);

        assertEquals(fetched.statusCode(), 200);
        assertEquals(fetched.jsonPath().getString("customerId"), id);
    }
}
```

INTERVIEW TIP
If the endpoint path changes, you edit one class, not 200 tests. That single sentence is the case for the service-object layer.

40 Where do you keep the base URI, authentication and common headers?

SHORT ANSWER
In a request-specification factory. The base URI comes from configuration, the token from the TokenManager, and the common headers are set once in the builder.

```
public final class RequestSpecFactory {

    private RequestSpecFactory() { }

    private static RequestSpecBuilder baseBuilder() {
        return new RequestSpecBuilder()
            .setBaseUri(ConfigReader.get("baseUrl"))
            .setContentType(ContentType.JSON)
            .setAccept(ContentType.JSON)
            .addFilter(new ApiLogFilter());
    }

    public static RequestSpecification anonymous() {
        return baseBuilder().build();
    }

    public static RequestSpecification authenticated() {
        return asUser(TokenManager.getToken());
    }

    public static RequestSpecification asUser(String token) {
        return baseBuilder().addHeader("Authorization", "Bearer " + token).build();
    }
}
```

What	Where it lives
Base URI, timeouts, tenant, country	Environment properties (config-qa.properties, ...)
Client secrets and passwords	Environment variables or the CI credential store, never Git
Common headers and content type	RequestSpecFactory
Access token	TokenManager (cached, refreshed, thread-safe)
Endpoint paths	API client classes

41 How do you create a reusable RequestSpecification?

SHORT ANSWER
Build it once with RequestSpecBuilder, expose named variants (anonymous, authenticated, as another user), and add extras per test with additional spec() calls.

```
// anonymous, for testing 401 behaviour
given().spec(RequestSpecFactory.anonymous()).when().get("/accounts").then().statusCode(401);

// a second user, for authorization tests
given().spec(RequestSpecFactory.asUser(userBToken)).when().get("/accounts");

// specs can be combined: later settings add to or override earlier ones
RequestSpecification tenant = new RequestSpecBuilder().addHeader("X-Tenant", "IN").build();
given().spec(RequestSpecFactory.authenticated()).spec(tenant)
.when().get("/orders");
```

WATCH OUT

Avoid `RestAssured.baseURI = ...` and other static setters for per-test values. They are global, so they cause cross-test interference when tests run in parallel. Reserve static configuration for genuinely global settings such as logging and timeouts.

42 How do you create a reusable ResponseSpecification?

SHORT ANSWER

Capture the checks every response of a kind should pass (status, content type, error structure) in named specs, and add test-specific assertions on top.

```
public final class ResponseSpecs {

    private ResponseSpecs() { }

    public static ResponseSpecification ok() {
        return new ResponseSpecBuilder()
            .expectStatusCode(200)
            .expectContentType(ContentType.JSON)
            .build();
    }

    public static ResponseSpecification created() {
        return new ResponseSpecBuilder()
            .expectStatusCode(201)
            .expectContentType(ContentType.JSON)
            .build();
    }

    public static ResponseSpecification error(int status) {
        return new ResponseSpecBuilder()
            .expectStatusCode(status)
            .expectContentType(ContentType.JSON)
            .expectBody("errorCode", notNullValue())
            .expectBody("message", not(emptyOrNullString()))
            .build();
    }
}

given().spec(requestSpec).body(payload)
.when().post("/customers")
.then().spec(ResponseSpecs.created())
       .body("status", equalTo("SUCCESS"));
```

INTERVIEW TIP

Be careful with response-time checks in a shared spec: a slow CI agent would then fail every test. Keep SLA checks in a separate spec used only by the tests that mean it.

43 How do you manage DEV, QA, UAT and PROD?

SHORT ANSWER

One codebase and one property file per environment, selected by a command-line parameter, with safeguards so that only safe tests can ever run against production.

Concern	Approach
Selecting the environment	mvn clean test -Denv=uat, mapped to a Jenkins choice parameter.
URLs and settings	config-dev / qa / uat / prod .properties loaded by ConfigReader.
Secrets	Environment variables or Jenkins credentials injected at runtime.
Environment-specific data	Test data files or generator rules keyed by environment.
Production safety	Only read-only smoke tests (a TestNG group) may run against prod. Enforce this in code.

```
public abstract class BaseTest {

    @BeforeSuite(alwaysRun = true)
    public void globalSetup() {
        String env = System.getProperty("env", "qa");
        String groups = System.getProperty("groups", "");
        if ("prod".equalsIgnoreCase(env) && !"smoke".equals(groups)) {
            throw new IllegalStateException(
                "Only the read-only 'smoke' group may run against production");
        }

        RestAssured.enableLoggingOfRequestAndResponseIfValidationFails();
        RestAssured.config = RestAssuredConfig.config().httpClient(
            HttpClientConfig.httpClientConfig()
                .setParam("http.connection.timeout", 10_000)
                .setParam("http.socket.timeout", 30_000));
    }
}
```

```
mvn clean test -Denv=qa -Dgroups=regression
mvn clean test -Denv=prod -Dgroups=smoke
```

CHAPTER 14

CI/CD, Logging and Reporting

Scenarios 44 to 46: tests only protect the product when they run automatically and explain their failures.

44 How do you integrate Rest Assured with Jenkins?

SHORT ANSWER

Jenkins checks out the code, runs Maven with the environment and suite as parameters, publishes JUnit and HTML reports, and notifies the team.




```
pipeline {
    agent any

    parameters {
        choice(name: 'ENV',   choices: ['qa', 'uat'],           description: 'Environment')
        choice(name: 'SUITE', choices: ['smoke', 'regression'], description: 'TestNG group')
    }

    tools {
        maven 'Maven-3.9'
        jdk   'JDK-17'
    }

    stages {
        stage('Checkout') {
            steps { checkout scm }
        }
        stage('API tests') {
            steps {
                withCredentials([string(credentialsId: 'api-client-secret',
                                        variable: 'CLIENTSECRET')]) {
                    sh "mvn -B clean test -Denv=${params.ENV} -Dgroups=${params.SUITE}"
                }
            }
        }
    }

    post {
        always {
            junit 'target/surefire-reports/*.xml'
            publishHTML(target: [reportDir: 'reports', reportFiles: 'extent.html',
                                 reportName: 'API Test Report', keepAll: true])
        }
        failure {
```

```
            mail to: 'qa-team@example.com',
                 subject: "API tests failed: ${env.JOB_NAME} #${env.BUILD_NUMBER}",
                 body: "Details: ${env.BUILD_URL}"
        }
    }
}
```

> **NOTE**
> Tool names such as Maven-3.9 and JDK-17 must match the names configured in your Jenkins. The publishHTML step needs the HTML Publisher plugin.

The same idea in GitHub Actions:

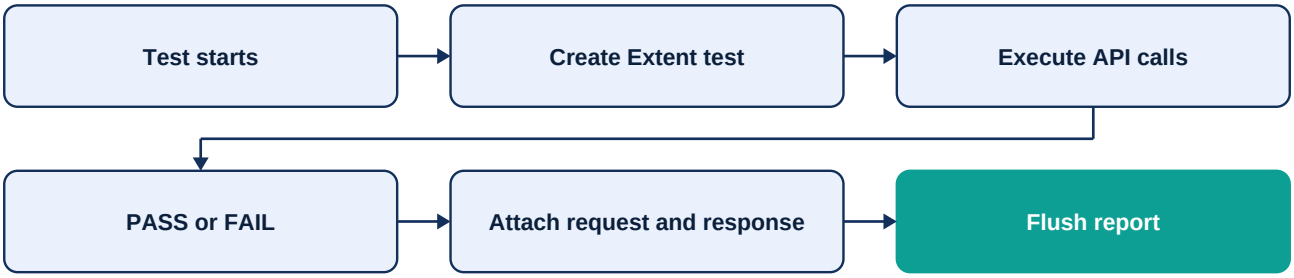
```yaml
name: api-tests
on:
  push:
    branches: [main]
  schedule:
    - cron: "0 2 * * *"        # nightly regression

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: "17"
          cache: maven
      - run: mvn -B clean test -Denv=qa -Dgroups=regression
        env:
          CLIENTSECRET: ${{ secrets.CLIENT_SECRET }}
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: api-test-report
          path: reports/
```

- Run a small **smoke** group on every deployment and the full **regression** nightly.
- Fail the build on test failure so a broken build is visible to everyone.
- Keep secrets in the CI credential store and mask them in logs.
- Publish JUnit XML (for trend graphs) and an HTML report (for humans).

45 How do you add Extent Reports?

> **SHORT ANSWER**
> Register a TestNG listener that creates an Extent test per test method, records pass or fail, attaches the request and response evidence, and flushes the report at the end.




```
public class TestListener implements ITestListener {

    private static final ExtentReports EXTENT = createReport();
    private static final ThreadLocal<ExtentTest> TEST = new ThreadLocal<>();

    private static ExtentReports createReport() {
        ExtentSparkReporter spark = new ExtentSparkReporter("reports/extent.html");
        spark.config().setReportName("API Automation Report");
        ExtentReports extent = new ExtentReports();
        extent.attachReporter(spark);
        extent.setSystemInfo("Environment", System.getProperty("env", "qa"));
        return extent;
    }

    @Override public void onTestStart(ITestResult r) {
        TEST.set(EXTENT.createTest(r.getMethod().getMethodName()));
    }

    @Override public void onTestSuccess(ITestResult r) {
        TEST.get().pass("Passed");
        ApiLogFilter.drain();                    // discard evidence for passing tests
    }

    @Override public void onTestFailure(ITestResult r) {
        TEST.get().fail(r.getThrowable());
        TEST.get().info("<pre>" + ApiLogFilter.drain() + "</pre>");
    }

    @Override public void onFinish(ITestContext ctx) {
        EXTENT.flush();
    }
}
```

Register it once in `testng.xml` with `<listeners><listener class-name="..."/></listeners>`. A ThreadLocal keeps parallel tests from writing into each other's report entries.

46 How do you capture the request and response when a test fails?

SHORT ANSWER

Use log().ifValidationFails() for simple cases. In a framework, add a Rest Assured filter that records every call and attach the record to the report only for failures.

```
given().log().all()
.when().get("/customer")
.then().log().ifValidationFails().statusCode(200);
```

```
public class ApiLogFilter implements Filter {

    private static final ThreadLocal<StringBuilder> LOG =
        ThreadLocal.withInitial(StringBuilder::new);

    @Override
    public Response filter(FilterableRequestSpecification req,
                           FilterableResponseSpecification res,
                           FilterContext ctx) {
        Response response = ctx.next(req, res);          // execute the call

        Object body = req.getBody();
        LOG.get()
           .append(req.getMethod()).append(' ').append(req.getURI()).append('\n')
           .append("Request body: ").append(body).append('\n')
           .append("Status: ").append(response.statusLine()).append('\n')
           .append("Response: ").append(response.asPrettyString()).append("\n\n");
        return response;
    }

    public static String drain() {
        String text = LOG.get().toString();
        LOG.remove();
        return text;
    }
}
```

WATCH OUT

Never write the Authorization header, passwords or tokens into reports. The filter above deliberately omits request headers. If you add them, redact sensitive values first.

INTERVIEW TIP

Say why you capture evidence only for failures: reports stay small and readable, and no secrets are written for the thousands of tests that pass.

CHAPTER 15

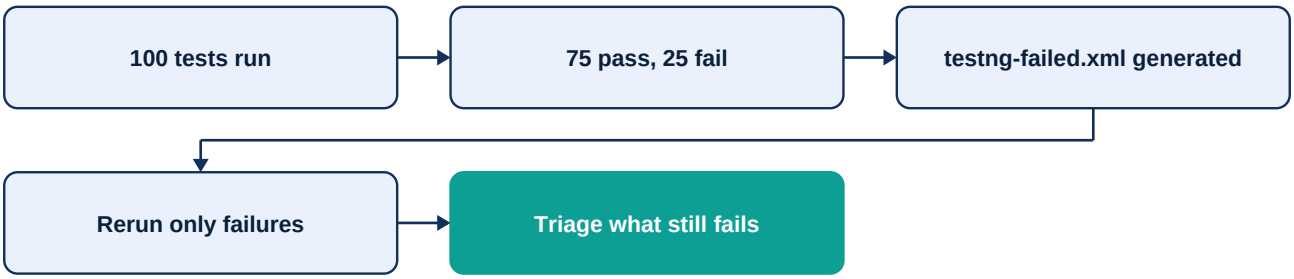
Senior-Level Scenarios

Scenarios 47 to 50: flaky failures, reruns and slow suites. This is where interviews move from tools to judgement.

47 You ran 100 tests and 25 failed. How do you run only the failed ones?

SHORT ANSWER

Use the testng-failed.xml that TestNG generates, or Surefire's rerun option. Then triage before rerunning again, because reruns can hide real defects.




```
# TestNG writes the file to the reports folder after a run
mvn test -Dsurefire.suiteXmlFiles=target/surefire-reports/testng-failed.xml

# Or let Surefire rerun failures inside the same build; passing reruns are reported as "flaky"
mvn test -Dsurefire.rerunFailingTestsCount=2
```

A TestNG retry analyzer gives you finer control, for example retrying only tests that opt in:

```
public class RetryAnalyzer implements IRetryAnalyzer {

    private static final int MAX_RETRIES = 2;
    private int attempt = 0;

    @Override
    public boolean retry(ITestResult result) {
        return attempt++ < MAX_RETRIES;      // true means: run the test again
    }
}

@Test(retryAnalyzer = RetryAnalyzer.class)
public void searchOrders() { ... }
```

Failure type	How to recognise it	Action
Product defect	Fails every time; response contradicts the requirement	Raise a bug with evidence.
Test defect	Fails every time; the test expectation is wrong or outdated	Fix the test.

Failure type	How to recognise it	Action
Data issue	Depends on data state, or passes on a fresh environment	Make tests create their own data.
Environment issue	5xx, timeouts or connection errors across many tests	Check environment health; rerun later.
Flaky test	Passes on rerun without any change	Find the race or dependency; do not hide it.

48 An API randomly returns 500. How do you handle a flaky API?

SHORT ANSWER

Investigate the root cause first. Add a limited, visible retry only where it is safe, and never let retries hide a genuine defect.

Check	Question to answer
Server logs	Is there an exception for that request ID?
Request data	Does it fail only for certain payloads or values?
Environment	Was there a deployment, restart or low-resource event?
Dependencies	Is a database or downstream service timing out?
Concurrency	Does it fail only when tests run in parallel (locks, shared data)?
Rate limiting	Are you being throttled and getting the wrong error code?
Network	Is the failure through a proxy, VPN or load balancer only?

```java
public static Response getWithRetry(Supplier<Response> call, int maxAttempts) {
    Response last = null;
    for (int attempt = 1; attempt <= maxAttempts; attempt++) {
        last = call.get();
        if (last.statusCode() < 500) {
            return last;
        }
        System.out.printf("Attempt %d/%d returned %d%n",
                          attempt, maxAttempts, last.statusCode());
        try {
            Thread.sleep(500L * attempt);                    // simple back-off
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
            break;
        }
    }
    return last;
}
```

- **Retry only safe calls.** Repeating a GET is harmless. Repeating a payment POST without an idempotency key is not.
- **Cap the attempts and log every one.** The first failure is evidence. Keep it in the report.
- **Track flakiness.** A test that needs retries regularly is a defect in the product or the test.
- **A 500 is a bug candidate even if the retry passes.** Report it with the request ID.

- **For asynchronous work, poll, do not sleep.** Use Awaitility with a timeout instead of a fixed Thread.sleep().

49 It works in Postman but fails in Rest Assured. How do you debug it?

SHORT ANSWER

Print the exact Rest Assured request with log().all() and compare it with Postman's request, character by character: URL, method, headers, auth, body and encoding.

```
given().log().all()          // shows URI, headers, cookies and body as actually sent
...
.then().log().all();         // shows the full response
```

Difference	What goes wrong	Fix
Authorization value	Token sent without the "Bearer " prefix, or an unresolved {{token}}	Use auth().oauth2(token) or the correct header.
Content-Type	Missing, or Rest Assured appends a charset that the server rejects (application/json; charset=UTF-8)	Set the content type explicitly; disable the automatic charset if needed (below).
Query encoding	An already-encoded value gets encoded twice	Pass raw values, or use urlEncodingEnabled(false).
Redirects	Postman follows a redirect that Rest Assured does not for this method	Check the status code, then redirects().follow(true).
Cookies and session	Postman keeps cookies between calls automatically	Pass cookies explicitly, or use a SessionFilter.
TLS certificate	Postman had verification switched off	Fix the trust store, or relaxedHTTPSValidation() in non-production only.
Hidden headers	Postman adds User-Agent and other default headers; some gateways expect them	Add the required header explicitly.
Proxy	Postman used a system proxy	Configure .proxy(host, port) to match.

```
// Do not append "; charset=UTF-8" to the Content-Type header automatically
RestAssured.config = RestAssuredConfig.config().encoderConfig(
    EncoderConfig.encoderConfig().appendDefaultContentCharsetToContentTypeIfUndefined(false));
```

INTERVIEW TIP

For stubborn cases, route both tools through the same proxy (Charles, Fiddler or mitmproxy) and diff the raw requests.

50 Your suite takes two hours. How do you bring it down?

SHORT ANSWER

Measure first, then attack the biggest costs: parallel execution, repeated authentication and setup, fixed sleeps and oversized data creation, and finally split the work across CI agents.

1. Find the bottleneck. Sort tests by duration from the report. A handful of slow tests usually account for most of the time.

2. Run independent tests in parallel.

```
<suite name="API Suite" parallel="methods" thread-count="5" data-provider-thread-count="5">
    <listeners>
        <listener class-name="com.example.api.listeners.TestListener"/>
    </listeners>
    <test name="Regression">
        <packages><package name="com.example.api.tests.*"/></packages>
    </test>
</suite>
```

3. Make the framework thread-safe before you switch on parallelism.

- No mutable `static` fields. Use ThreadLocal or per-test variables.
- A synchronized TokenManager, so one thread refreshes the token for all.
- Unique test data per test (UUID or timestamp), so tests never collide.
- Per-thread report entries and log buffers (as in Scenarios 45 and 46).

Optimisation	Why it saves time
Reuse the auth token	One token call instead of one per test.
Set up data through the API or database, not UI flows	Setup is a large share of runtime in many suites.
Replace Thread.sleep() with polling (Awaitility)	Wait only as long as the system needs.
Share read-only reference data	Avoid creating identical records repeatedly.
Split into smoke, sanity and regression groups	Fast feedback on every commit, the full suite nightly.
Shard across CI agents	Run groups or packages on several Jenkins agents at once.
Prune duplicates and low-value tests	Fewer tests, same coverage.

INTERVIEW TIP

Model answer: "I first identify the bottleneck from the reports. Then I parallelise independent tests, reduce redundant authentication and setup calls, optimise test data, remove unnecessary waits, and distribute suites across CI agents. I make sure shared utilities are thread-safe before enabling parallel execution."

PART V

Interview Toolkit

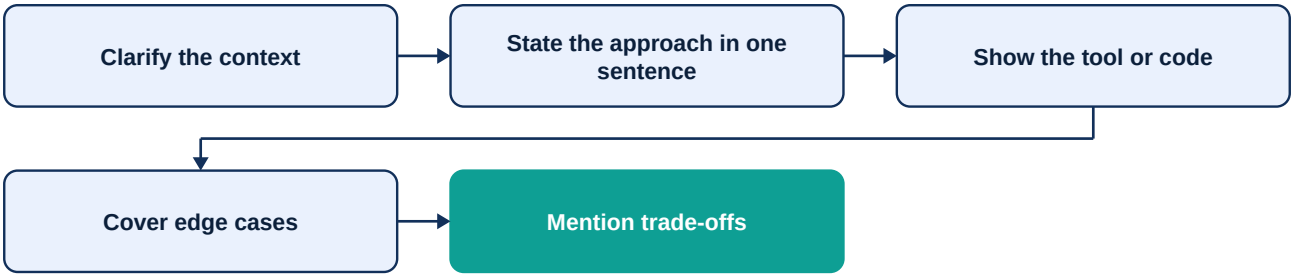
Whiteboard topics, rapid-fire questions, cheat sheets, templates and a glossary for the final days before the interview.

CHAPTER 16

The Interview Toolkit

Knowing the answers is half the job. The other half is structure and calm delivery.

A structure for any scenario question



For example, for "How would you handle token expiry?": clarify the lifetime and grant type, say "a cached token manager that refreshes shortly before expiry", sketch the code, mention thread-safety and a 401 fallback, and finish with the trade-off between refresh-ahead and refresh-on-401.

Ten topics to explain on a whiteboard

Topic	What to draw or say	Follow-up trap
1. RequestSpecification	Builder with base URI, content type, auth; reused via spec()	Why not static RestAssured.baseURI?
2. ResponseSpecification	Shared status, content type and error-body checks	What should not go in a shared spec?
3. API chaining	POST, extract ID, GET; the value lives in a local variable	How does it behave in parallel?
4. Token management	Cache, buffer before expiry, synchronized refresh	What if the token is revoked early?
5. POJO serialisation	Jackson maps objects to JSON both ways; builders for defaults	How do you send a JSON null?
6. JSON schema validation	Structure, types, required fields, enums	What does it not prove?
7. Dynamic payloads	Template file plus overrides; unique data per run	How do you avoid 50 nearly identical files?
8. Environment management	Property files, -Denv, secrets from CI	How do you protect production?
9. Failed-test rerun	testng-failed.xml, retry analyzer, triage	How do retries hide bugs?
10. Parallel execution	Thread-safety rules, ThreadLocal, unique data	What breaks first when you go parallel?

Rapid-fire questions

Question	Short answer
PUT vs PATCH?	PUT replaces the whole resource and is idempotent. PATCH changes only the fields you send.
PUT vs POST?	POST submits to a server-chosen URI (create or process) and is not idempotent. PUT writes to a client-known URI and is idempotent.
401 vs 403?	401: not authenticated. 403: authenticated but not allowed.
Path vs query parameter?	Path parameters identify a resource. Query parameters filter, sort or paginate.
What is idempotency?	Repeating a request leaves the server in the same state as sending it once. It makes retries safe.
What does stateless mean?	Each request carries all it needs. The server keeps no client session between calls.
REST vs SOAP?	REST is an architectural style, usually JSON over HTTP. SOAP is a protocol with an XML envelope and a WSDL contract.
200 vs 204?	200 returns a body. 204 succeeds with no body.
What is a contract test?	A test that checks a provider and its consumers agree on request and response shapes (schema checks, Pact).
How do you test pagination?	First, middle, last and beyond-last pages; size limits; consistent totals; no duplicates or gaps.
How do you test an async API (202)?	Poll the status endpoint with a timeout (Awaitility) until it reaches a final state, then verify the result.
Does CORS affect API tests?	No. It is enforced by browsers, not by Postman or Rest Assured. Test the CORS configuration itself separately.
How do you upload a file?	given().multiPart("file", new File(path)) with a multipart content type.
How do you send query parameters?	given().queryParam("status", "OPEN") or queryParams(...).
How do you convert a response to a POJO?	response.as(Customer.class), or .extract().as(...).
What does relaxedHTTPSValidation() do?	Skips certificate trust checks. Use it only in non-production test environments.
How do you test when a dependency is down?	Use a mock or virtual service (WireMock, MockServer) to simulate the dependency and its failures.
How do you verify the database after an API call?	Query it through JDBC in the test, or verify indirectly with a GET. Keep DB checks minimal.
What is HATEOAS?	Responses contain links that tell the client which actions are available next.
What is OpenAPI/Swagger used for?	A machine-readable contract: documentation, client and stub generation, and response validation.
What is rate limiting?	Restricting calls per time window, returning 429 with Retry-After when exceeded.
Where does the JWT payload get its security?	From the signature. The payload is only encoded, so anyone can read it and it must not hold secrets.

Final-week checklist

- Explain all ten whiteboard topics aloud in under two minutes each.
- Write a TokenManager, a RequestSpecFactory and a CRUD chain from memory.
- Prepare one real story each for a flaky test you fixed, a suite you sped up and a defect you found through API testing.
- Practise the 50 scenarios as spoken answers: one sentence first, then depth.
- Have a clear opinion on retries, parallelism, test data and environment safety.

APPENDIX A

Quick Reference

Commands and snippets to keep within reach.

Rest Assured cheat sheet

Task	Code
Base URI and content type	`given().baseUri(url).contentType(ContentType.JSON)`
Path / query parameter	`pathParam("id", 1) / queryParam("page", 2)`
Bearer token	`auth().oauth2(token)`
Basic auth (preemptive)	`auth().preemptive().basic(user, pass)`
Send a POJO / Map / file	`body(pojo) / body(map) / body(new File(path))`
Assert status and header	`statusCode(200).header("Content-Type", containsString("json"))`
Assert a JSON field	`body("customer.address.city", equalTo("Delhi"))`
Assert a collection	`body("users", hasSize(3)).body("users.id", hasItem(101))`
Response time	`time(lessThan(3000L))`
Schema validation	`body(matchesJsonSchemaInClasspath("schemas/x.json"))`
Extract one value	`extract().path("id")`
Extract as POJO	`extract().as(Customer.class)`
Log on failure	`log().ifValidationFails()`
Reuse specs	`given().spec(reqSpec) ... then().spec(respSpec)`
Ignore HTTPS trust (non-prod)	`given().relaxedHTTPSValidation()`
Follow / stop redirects	`redirects().follow(false)`

Command cheat sheet

```
# Maven + TestNG
mvn clean test                                  # everything
mvn clean test -Denv=uat                        # choose environment
mvn clean test -Dgroups=smoke                   # run a TestNG group
mvn test -Dtest=CustomerTests                   # one class
mvn test -Dtest=CustomerTests#createCustomer    # one method
mvn test -Dsurefire.suiteXmlFiles=testng.xml    # a specific suite file
mvn test -Dsurefire.rerunFailingTestsCount=2    # rerun failures in-build

# Newman
newman run collection.json -e qa.postman_environment.json -d data.csv

# cURL
curl -i -X GET "https://qa.example.com/customers/101" -H "Authorization: Bearer $TOKEN"
```

APPENDIX B

Templates

Copy these into your team wiki or test-management tool.

API test case template

Field	Content
ID and title	TC-CUST-001 Create customer with valid data
Endpoint	POST /customers
Preconditions	Valid token with customers.write scope; mobile number not yet registered
Headers	Content-Type: application/json; Authorization: Bearer <token>
Request body	{"name":"Ajay","mobile":"9876543210"}
Expected status	201 Created
Expected response	customerId present; status = SUCCESS; Location header set
Expected side effects	Customer retrievable by GET; audit record written; one welcome event published
Cleanup	DELETE /customers/{customerId}

Defect report template

Field	Content
Title	POST /customers returns 500 when mobile contains letters
Environment and build	UAT, build 2026.09.18-3, 22 Sep 2026 10:42 IST
Request ID	e7c1f3a2-0b5d-4c1e-9d55-0d1c6f3b9a11
Steps to reproduce	cURL command with secrets removed
Expected result	400 Bad Request with message "Mobile must contain digits only"
Actual result	500 Internal Server Error with a stack trace in the response body
Frequency	Always
Severity	High: server error and information disclosure

New-API readiness checklist

- Contract (OpenAPI) reviewed and questions answered.
- Test data and accounts (including two users and roles) available.
- Happy path, validation, auth, boundary, state and error-contract cases designed.
- Schema file versioned with the tests; smoke tests tagged for the pipeline.
- Performance target and rate limits known.

APPENDIX C

Glossary

Terms used in this book.

Term	Meaning
API	Application Programming Interface: a contract that lets one software system use another.
Endpoint	A URL and HTTP method combination that exposes an operation or resource.
Payload	The body of a request or response.
Idempotent	Repeating the request has the same effect on server state as sending it once.
Schema	A formal description of the structure and types of a JSON document.
POJO	Plain Old Java Object: a simple class used to model request or response data.
Serialisation	Converting an object to JSON (or another format). Deserialisation is the reverse.
BDD	Behaviour-driven style. In Rest Assured: given, when, then.
Hamcrest	A library of matchers such as equalTo and hasItem used in assertions.
GPath	Groovy's path language used by Rest Assured to navigate JSON and XML.
JWT	JSON Web Token: a signed, base64url-encoded token carrying claims.
OAuth 2.0	An authorization framework that issues short-lived access tokens.
BOLA / IDOR	Broken object-level authorization: accessing another user's object by changing an ID.
SLA	Service-level agreement: an agreed performance or availability target.
Flaky test	A test that passes and fails without any change to the code under test.
Smoke / regression suite	A small critical-path check, and the broader check that protects existing behaviour.
Service object	A class that wraps the calls of one API so tests do not repeat endpoint details.
Mock / virtual service	A stand-in for a dependency that returns controlled responses.
Contract test	A test that verifies provider and consumer agree on message formats.
Shift left	Testing earlier in the lifecycle to find defects when they are cheaper to fix.

WHAT NEXT

Keep Practising

A book gives you the map. Practice builds the muscle.

- Pick a public practice API, or your own project, and build the framework of Chapter 13 from scratch.
- Turn each of the 50 scenarios into a spoken 60-second answer, then a two-minute deep dive with code.
- Automate a full CRUD lifecycle, a token-expiry test and a two-user authorization test. These three come up again and again.
- Run your suite in a real pipeline, then make it faster with the ideas in Scenario 50.

For subject-wise interview questions across 22 SDET subjects (Manual Testing, Java, Selenium, Playwright, TestNG, API, Appium, JMeter, Python, SQL and more), scenario answers, coding-round practice, a mock interview mode and a layer-by-layer automation framework blueprint, visit **sdetajay.com**.

Think Switch, Think SDET AJAY.